

编译原理

11. 寄存器分配

rainoftime.github.io

**浙江大学
计算机科学与技术学院**

Content

1. Introduction
2. Lexical Analysis
3. Parsing
4. Abstract Syntax
5. Semantic Analysis
6. Activation Record
7. Translating into Intermediate Code
8. Basic Blocks and Traces
9. Instruction Selection
10. Liveness Analysis
- 11. Register Allocation**
13. Garbage Collection
14. Object-oriented Languages
18. Loop Optimizations

Outline

-  **Introduction to Register Allocation**
-  **Register Allocation via Graph Coloring**
-  **Coloring by Simplifications**
-  **Coloring by Simplifications, Cont.**

1. Introduction to Register Allocation

What is Register Allocation

- **The Problem**

- Programs use unlimited temporary variables (temps)
- Real machines have a limited number of registers
- Need to map temporaries to registers (efficiently)

- **The Goal**

- Map temporaries to registers
- Preserve program semantics
- Optimize performance

Register Allocation Algorithms

- **“Naïve” register allocation**
- **Local register allocation**
 - Basic block level
 - Does not capture reuse of values across multiple basic blocks
- **Global register allocation**
 - Function-level (for a whole function at a time; take the information of the function into consideration)
 - Linear scan or **graph-coloring** paradigm

Example: Registers Allocation in LLVM

- **Basic** : 线性扫描算法的改进，使用启发式的顺序对寄存器进行生存期赋值
- **Fast** : 顺序扫描每条指令，对其中的变量进行寄存器分配，当没有寄存器可以分配时，选择溢出代价最小的寄存器进行溢出操作
- **Greedy** : 线性扫描算法的改进，Basic分配器的高度优化的实现，合并了全局生存期分割
- **PBQP** : 基于分区布尔二次编程（PBQP）的寄存器分配器。构造一个表示寄存器分配问题的PBQP问题，使用PBQP求解器解决该问题

2. Register Allocation Via Graph Coloring: Overview

- **Interference Graph**
- **Register Allocation**

Register Allocation via Graph Coloring

- **Key insight:** Two temporaries can share a register if they are not live at the same time
- This is exactly a **graph coloring** problem:

Concept	Graph Coloring	Register Allocation
Node	Vertex	Virtual register / temporary
Edge	Adjacent	Two temps live at the same time
Color	Color class	Physical register

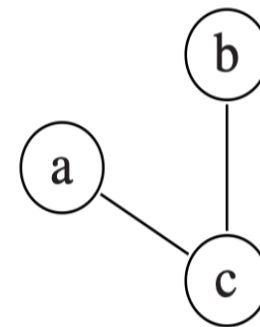
- **Goal:** Color the graph with K colors (where K is the number of available registers)

Interference Graph

- **Nodes** = virtual registers/temporaries
- **Edges** connect virtual registers that interfere (**live ranges overlap**)

	a	b	c
a			x
b			x
c	x	x	

(a) Matrix



(b) Graph

a matrix: x marking interferences

undirected graph

Two variables can share the same register if no edge connects them

Example: Interference graph

Instruction	Live-out
$b = a + 2$	$\{a, b\}$
$c = b * b$	$\{a, c\}$
$b = c + 1$	$\{a, b\}$
return $b * a$	$\{ \}$

color	register
	eax
	ebx

1. $b = a + 2$ defines **b**, live-out = $\{a, b\}$

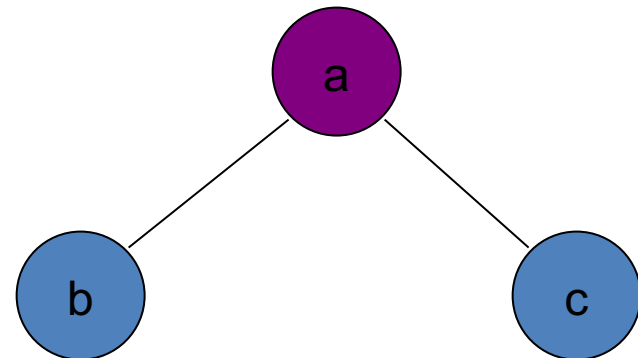
→ edge **(b, a)**

2. $c = b * b$ defines **c**, live-out = $\{a, c\}$

→ edge **(c, a)**

3. $b = c + 1$ defines **b**, live-out = $\{a, b\}$

→ edge (b, a) *exists*, plus **(b, c)**?



Build Interference Graphs

- For each instruction that **defines** variable a , with live-out set $out[n] = \{b_1, \dots, b_j\}$:

Case 1. (Non-move): If it is a non-move

add interference edges $(a, b_1), \dots, (a, b_j)$.

i.e., the defined var. interferes with *every* live-out var.

Case 2. (Move) If it is a move $a \leftarrow c$:

add interference edges (a, b_i) for any $b_i \neq c$

i.e., we *skip* the source c — this enables **move coalescing** (next slide).

Why the difference? After $a \leftarrow c$, variables a and c hold the *same value*. If we don't add edge (a, c) , the allocator may assign them the same register, making the move **redundant**.

Example: Move Coalescing

Before coalescing

```
t := s           // copy (move)
...
x := ... s ...   // use of s
...
y := ... t ..    // use of t
```

- s and t are **both live** after the copy
- Normally $\rightarrow$ interference edge (s, t)
- But they hold the *same value*!

After coalescing

```
// t := s        eliminated
...
x := ... s ...   // use of s
...
y := ... s ...   // t replaced by s
```

- If s and t get the **same color**, the copy is redundant
- We may delete it and replace all uses of t with s

Move coalescing = merging two nodes connected by a move edge into one, eliminating the copy instruction.

Example: When Coalescing Fails

```
t := s          // copy (move)
...
x := ... s ...  // use of s
...
y := ... t ..   // use of t
```

```
t := s  // copy
t := ... // later (non-move) def of t
x := ... s .. // use of s (s still alive)
...
y := ... t ... // use of t
```

- After the **second definition** of *t* (non-move), *t* gets a *new* value
- At that point, ***s* is still live**
- Now *t* and *s* genuinely interfere → **edge (*t*, *s*)** is added by Case 1
- They *cannot* share a register anymore

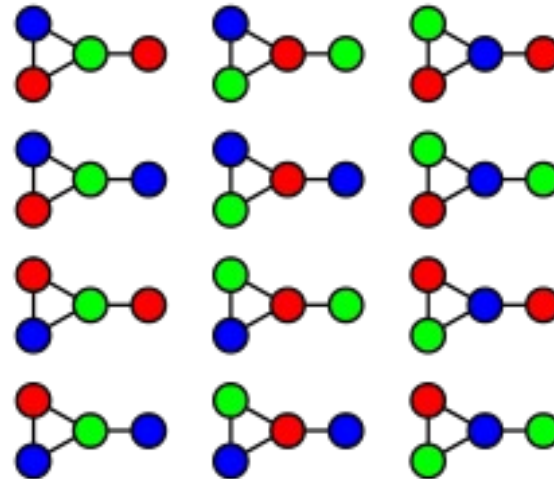
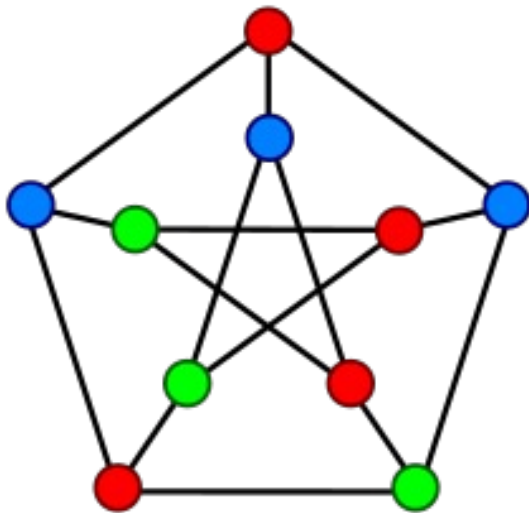
The move-special-case only applies at the *move instruction itself*.
If a later non-move instruction redefines the destination while the source is still live, the normal rule kicks in and creates the interference edge.

2. Register Allocation Via Graph Coloring: Overview

- **Interference Graph**
- **Register Allocation**

Graph Coloring

- **Vertex Coloring:** assign a color to each vertex such that no edge connects vertices with the same color.
- **K-Coloring:** a coloring using at most k colors



K-Coloring for Register Allocation

1. From live ranges construct an **interference graph**
2. Color the graph so that **no two neighbors have the same color**
3. If graph needs more than k colors – **Spilling**

Instructions	Live vars
--------------	-----------

	a
--	---

b = a + 2	
-----------	--

	a,b
--	-----

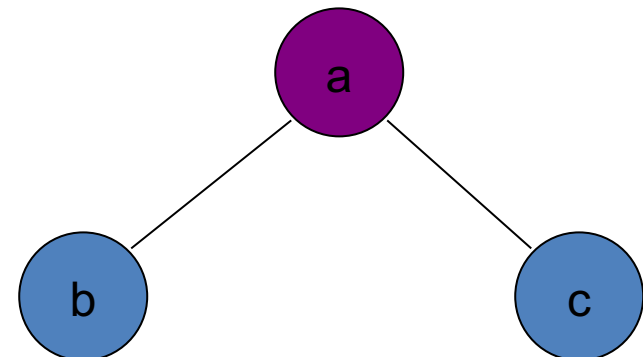
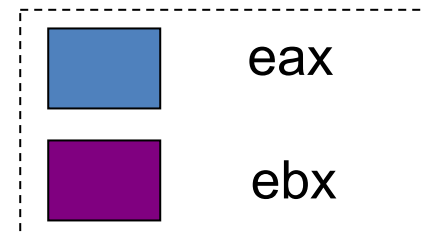
c = b * b	
-----------	--

	a,c
--	-----

b = c + 1	
-----------	--

	a,b
--	-----

return b * a	
--------------	--



How Difficult is Graph Coloring

Consider the following two different problems:

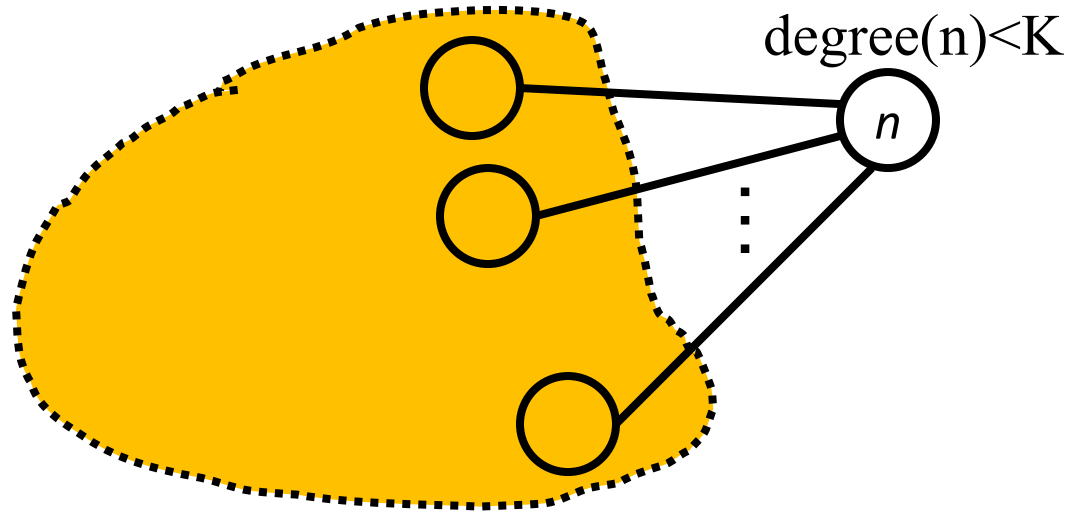
- 1. Find the least k such that the graph is k -colorable**
 - NP-hard
- 2. K-coloring: Given a constant k , decide whether the graph is k -colorable**
 - **NP-complete** (the problem we usually deal with in register allocation)
 - So, heuristics are needed

3. Coloring By Simplifications

- **Simplification & Select**
- **Simplification, Select, Spilling**

Kempe's Theorem

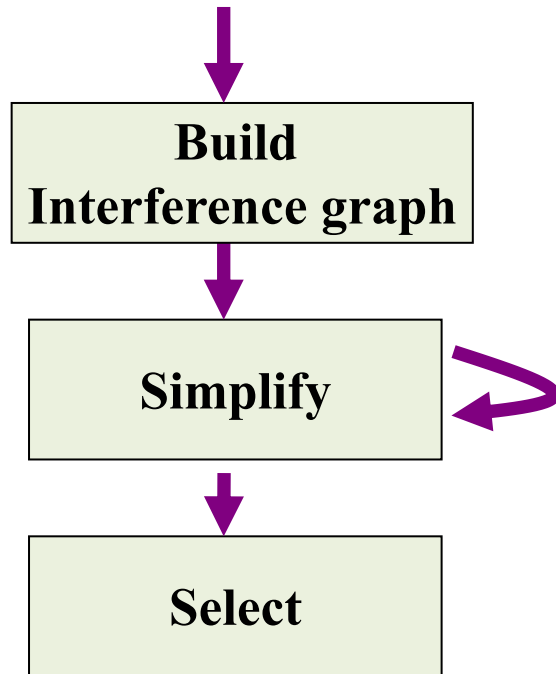
- [Kempe] Suppose the graph G contains a node n with fewer than K neighbors ($\text{degree}(n) < K$)
 - Let G' be the graph $G - \{n\}$ obtained by removing n and all edges connect n
 - If G' can be colored, then so can G (Why?)



- This lead naturally to a *stack-based* algorithm for coloring

Coloring by Simplification: Simplify & Select

Let's start with a simple algorithm with three ingredients



// Phase 1: Simplify

```
while graph G has node N with degree less than k
    Remove N and its edges from G
    push N on a stack S
```

// Phase 2: Select

```
if all nodes removed then graph is k-colorable
    while stack S contains node N
        Add N to graph G and assign it a color
```

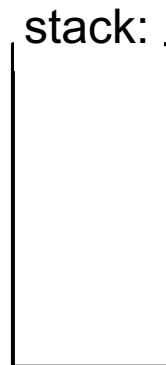
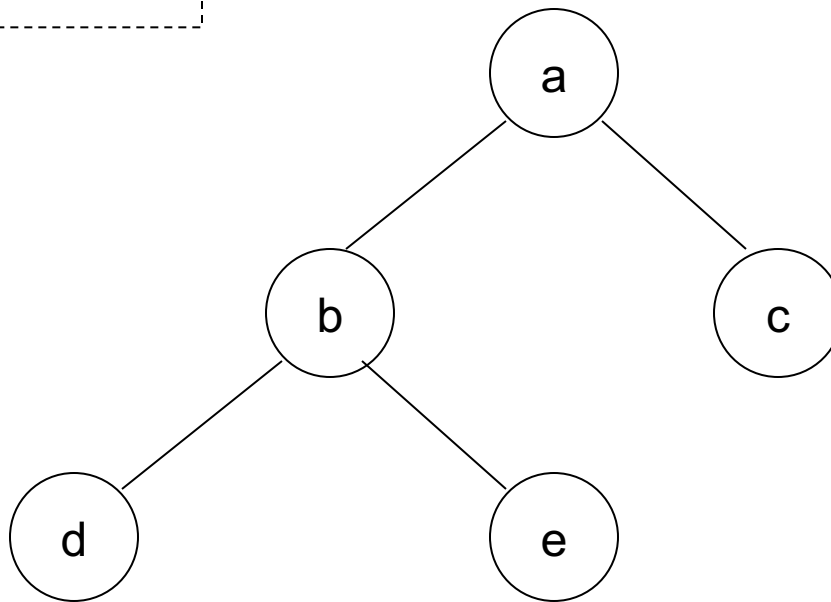
1. build
2. simplify
3. select (or color)

the conflict graph from the program
the nodes with insignificant degree
while rebuilding the graph.

Example: Simplification (Assume $K = 2$)

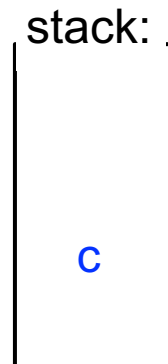
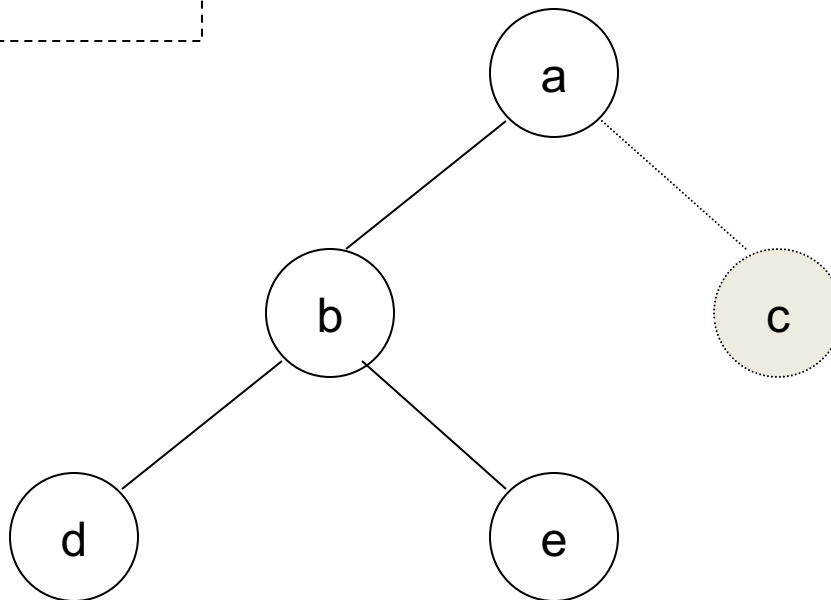
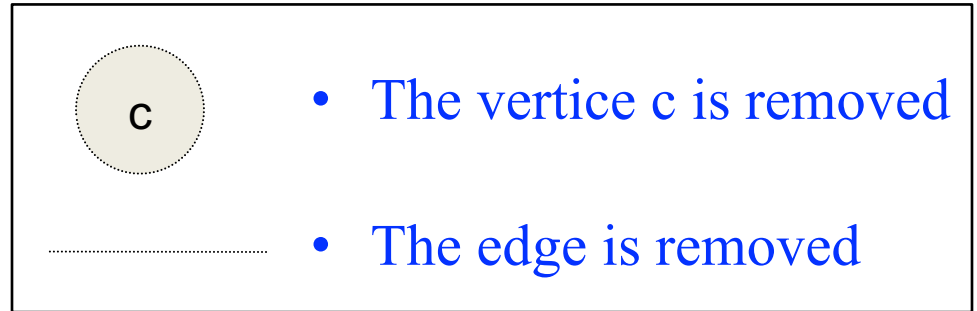
color	register
	eax
	ebx

- A vertex such that its **degree** $< k$ is always **k-colorable**
- Remove such vertices and push them to a stack **until the graph becomes empty**



Example: Simplification (Assume $K = 2$)

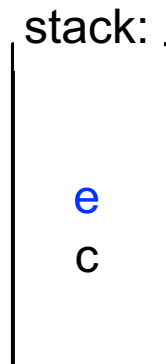
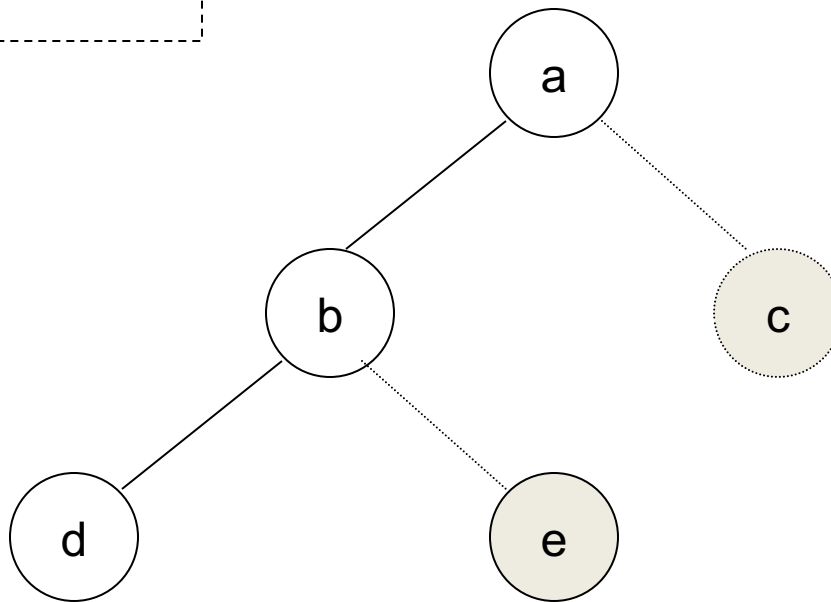
color	register
	eax
	ebx



Example: Simplification (Assume $K = 2$)

color	register
	eax
	ebx

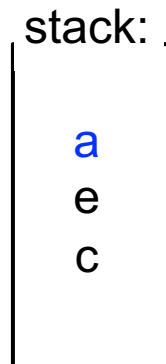
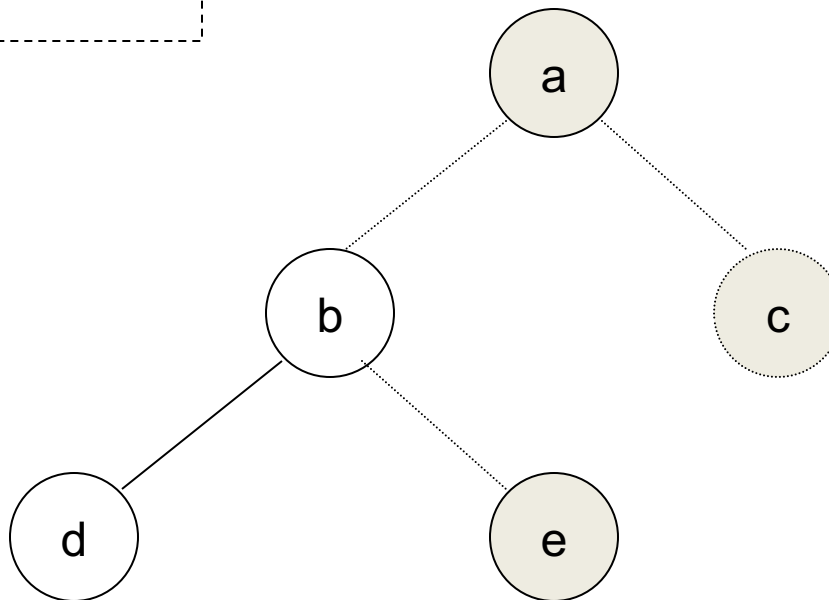
- A vertex such that its degree $< k$ is always k -colorable
- Remove such vertices and push them to a stack until the graph becomes empty



Example: Simplification (Assume $K = 2$)

color	register
	eax
	ebx

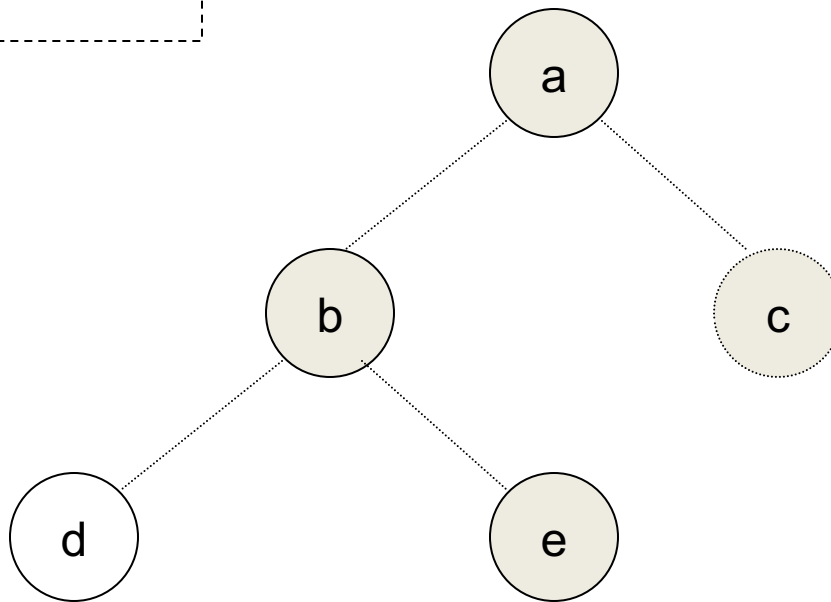
- A vertex such that its degree $< k$ is always k -colorable
- Remove such vertices and push them to a stack until the graph becomes empty



Example: Simplification (Assume $K = 2$)

color	register
	eax
	ebx

- A vertex such that its degree $< k$ is always k -colorable
- Remove such vertices and push them to a stack until the graph becomes empty

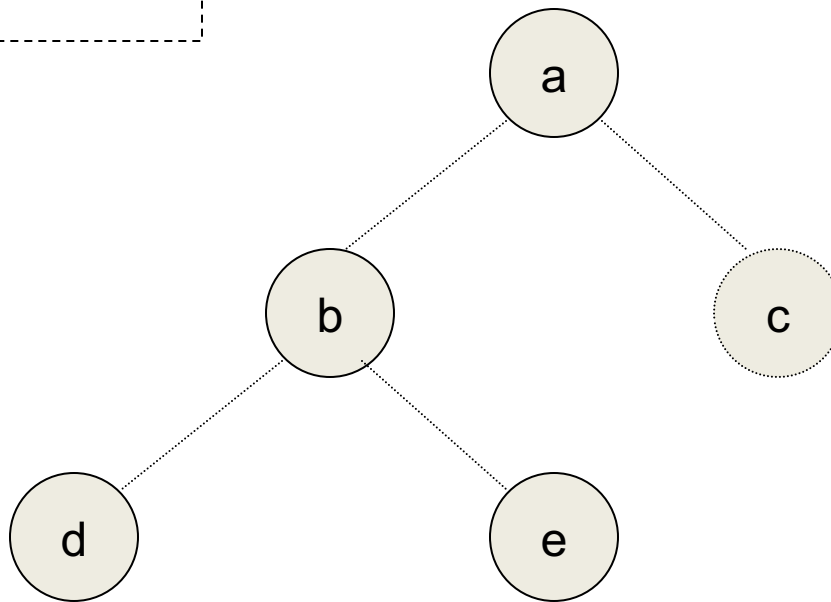


stack:
b
a
e
c

Example: Simplification (Assume $K = 2$)

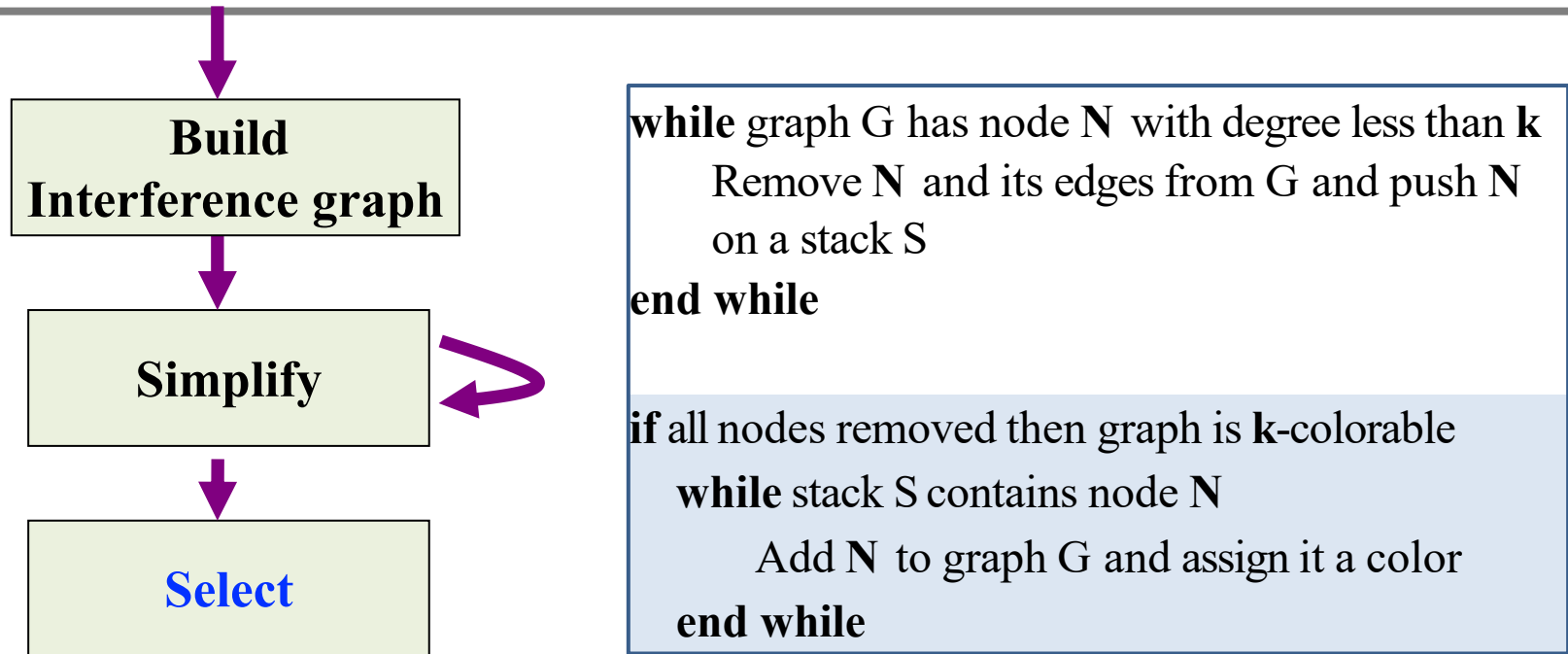
color	register
	eax
	ebx

- A vertex such that its degree $< k$ is always k -colorable
- Remove such vertices and push them to a stack **until the graph becomes empty!**



stack:
d
b
a
e
c

Select

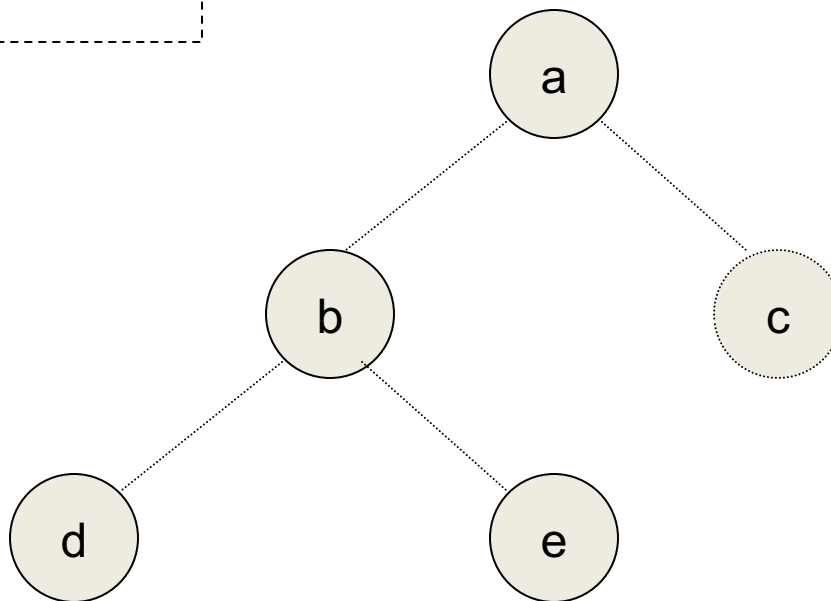


- **The simplification works (graph becomes empty)**
- **We can start assigning colors to the nodes!**
 - Starting with the empty graph
 - Rebuild the original graph by repeatedly adding a node from the top of the stack
 - When adding a node, there must be a color for it

Example: Select (Assume $K = 2$)

- Rebuild and color the graph!

color	register
	eax
	ebx

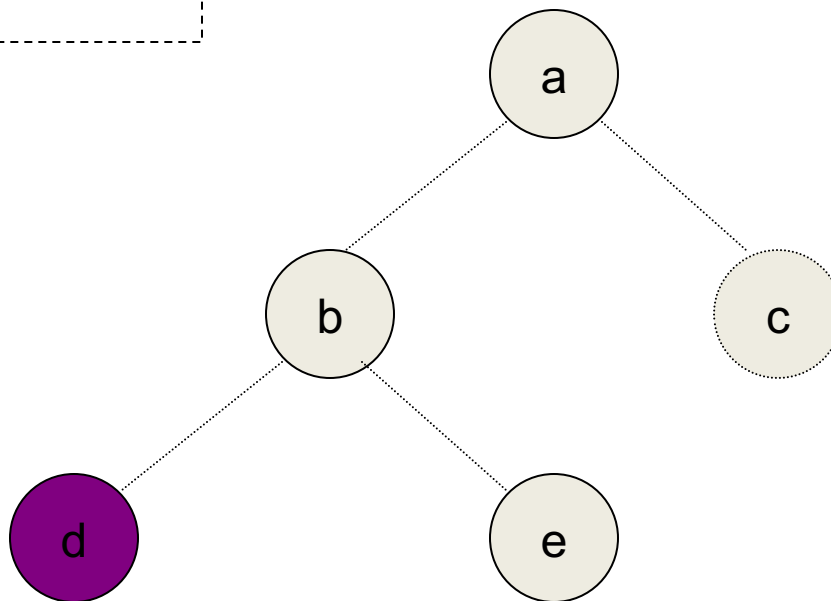


stack:
d
b
a
e
c

Example: Select (Assume $K = 2$)

- Rebuild and color the graph!

color	register
	eax
	ebx



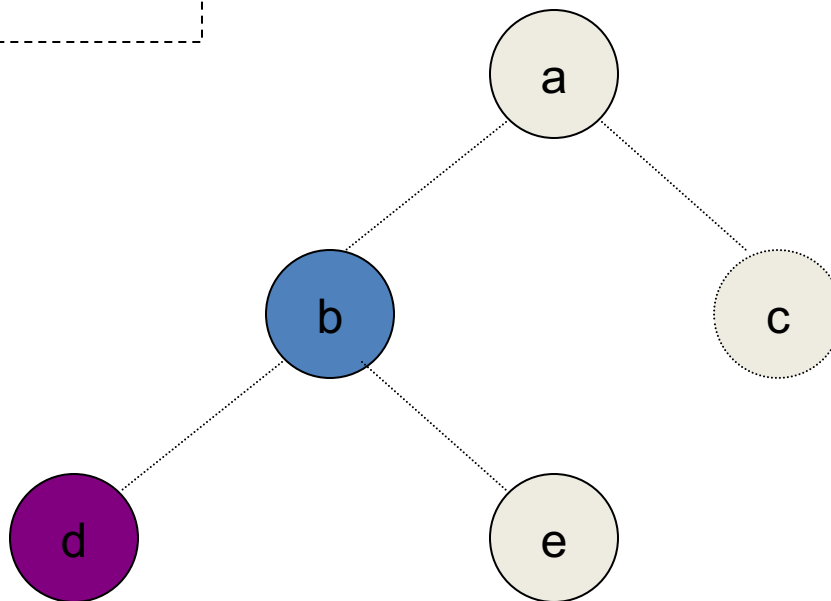
stack:

b
a
e
c

Example: Select (Assume $K = 2$)

- Rebuild and color the graph!

color	register
	eax
	ebx



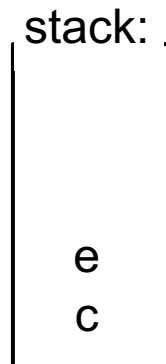
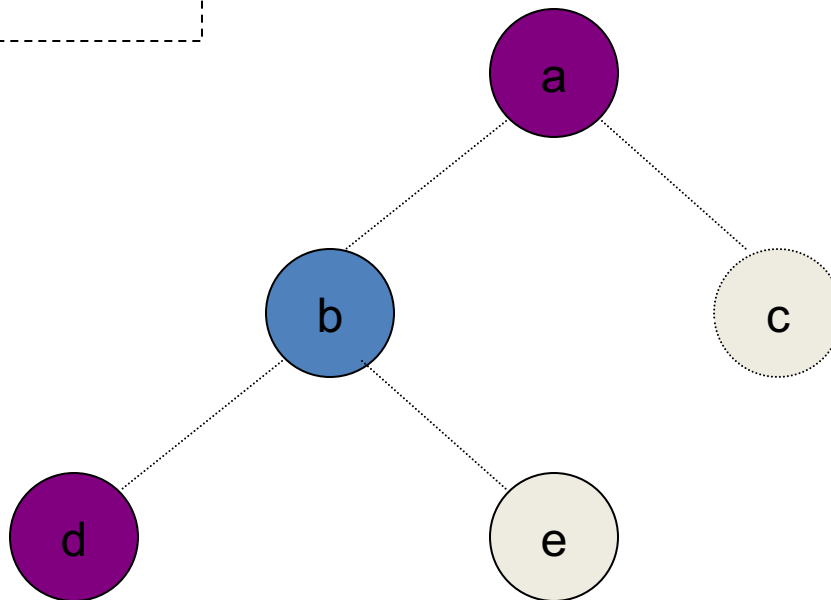
stack:

a
e
c

Example: Select (Assume $K = 2$)

- Rebuild and color the graph!

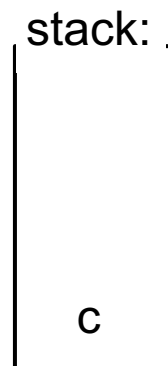
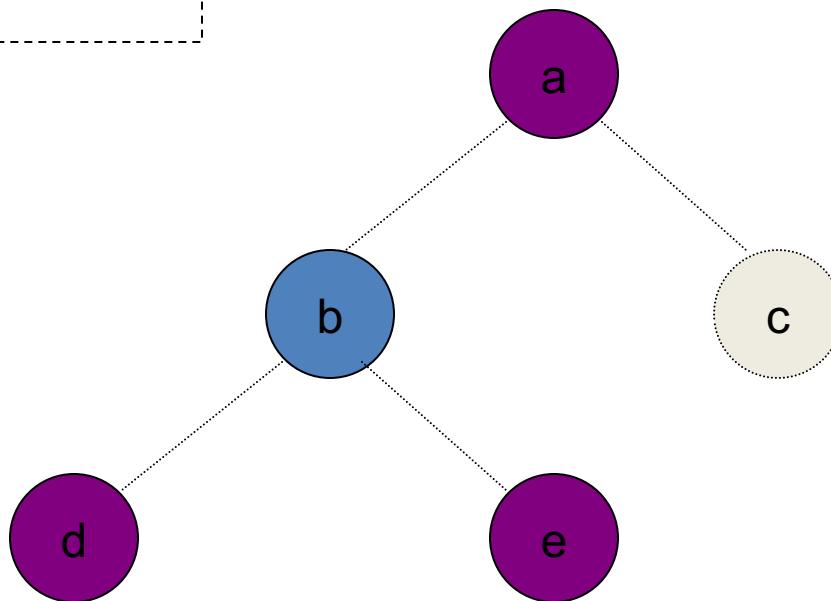
color	register
	eax
	ebx



Example: Select (Assume $K = 2$)

- Rebuild and color the graph!

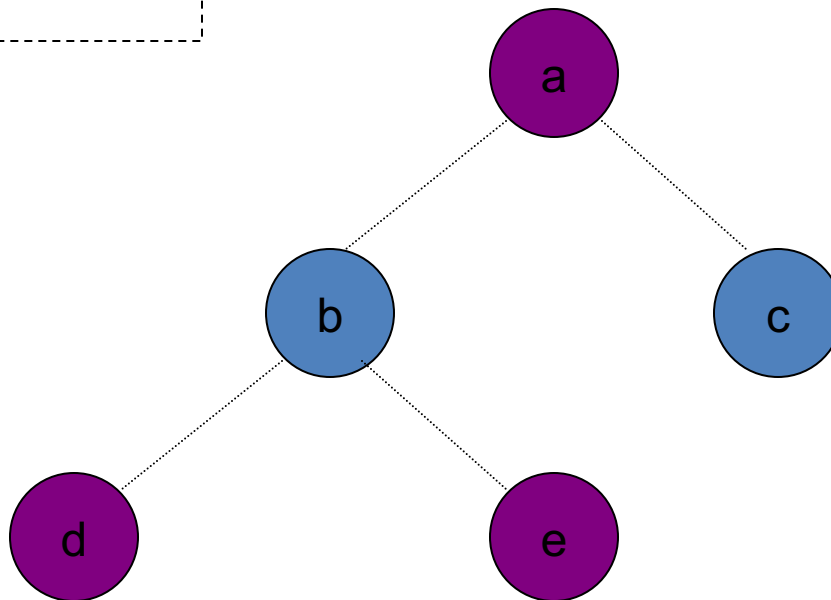
color	register
	eax
	ebx



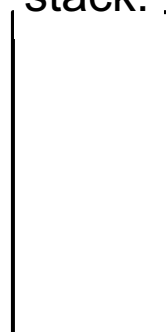
Example: Select (Assume $K = 2$)

- Rebuild and color the graph!

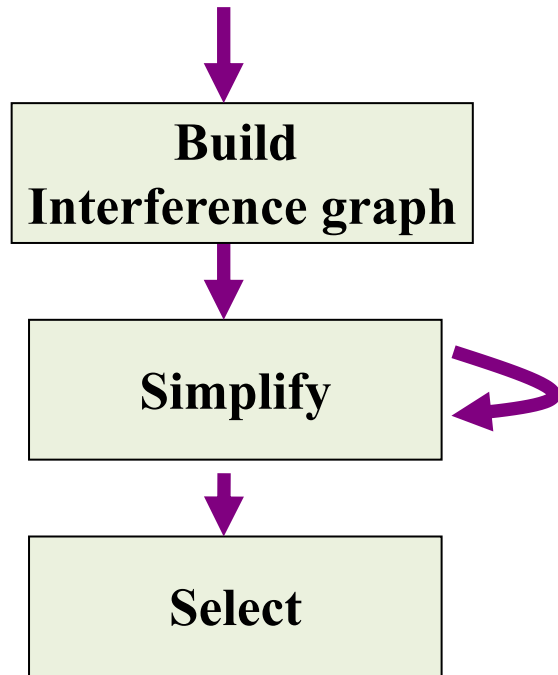
color	register
	eax
	ebx



stack:



Summary: Simplification & Select

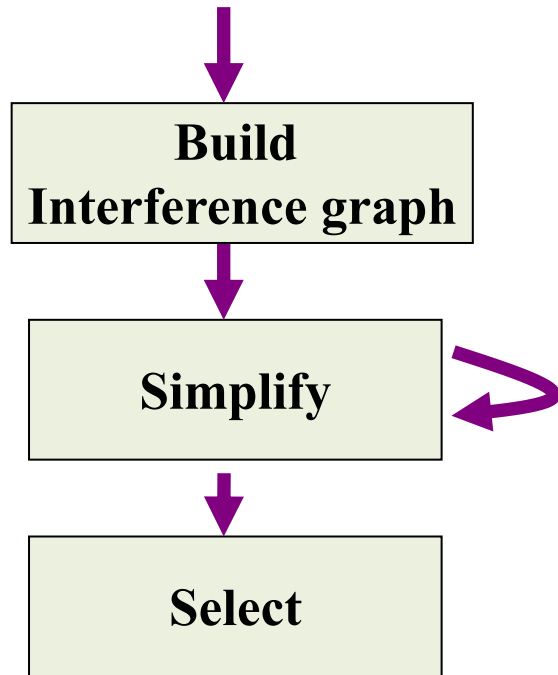


```
while graph G has node N with degree less than k  
    Remove N and its edges from G and push N  
    on a stack S  
end while  
  
if all nodes removed then graph is k-colorable  
    while stack S contains node N  
        Add N to graph G and assign it a color  
    end while
```

1. build
2. simplify
3. select (or color)

the conflict graph from the program
the nodes with insignificant degree
while rebuilding the graph.

Summary: Simplification & Select



```
while graph G has node N with degree less than k  
    Remove N and its edges from G and push N  
    on a stack S  
end while  
  
if all nodes removed then graph is k-colorable  
    while stack S contains node N  
        Add N to graph G and assign it a color  
    end while
```

1. build the conflict graph from the program
2. simplify the nodes with insignificant degree
3. select (or color) while rebuilding the graph.

- The algorithm is just a fast heuristic that **may fail!**
- When failed, it does not mean the graph is not k-colorable !

3. Coloring By Simplifications

- **Simplification & Select**
- **Simplification, Select, Spilling**
 - **Optimistic Coloring**
 - **Do the Actual Spill**

The “Simplification → Select” Heuristic May Fail

- At some point, the graph G has nodes only of **significant degree** (i.e., nodes of degree $\geq K$).
- Then, the previous algorithm does not work!

```
while graph  $G$  has node  $N$  with degree less than  $k$   
    Remove  $N$  and its edges from  $G$  and push  $N$  on a stack  $S$   
end while
```

```
if all nodes removed then graph is  $k$ -colorable  
    while stack  $S$  contains node  $N$   
        Add  $N$  to graph  $G$  and assign it a color from  $k$  colors  
    end while
```



One Possible Solution: Spilling

- At some point, the graph G has nodes only of **significant degree** (i.e., nodes of degree $\geq K$).
- Then, the previous algorithm does not work!

```
while graph  $G$  has node  $N$  with degree less than  $k$   
    Remove  $N$  and its edges from  $G$  and push  $N$  on a stack  $S$   
end while
```

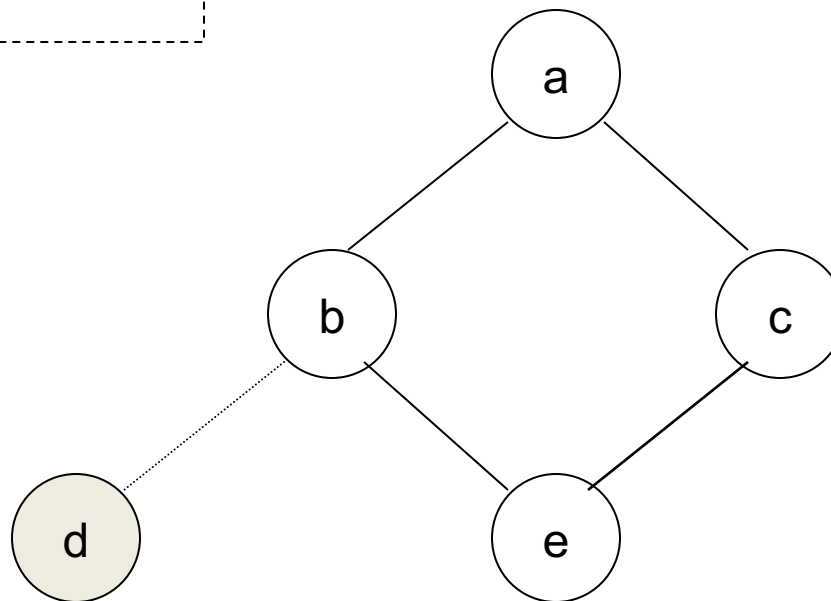
```
if all nodes removed then graph is  $k$ -colorable  
    while stack  $S$  contains node  $N$   
        Add  $N$  to graph  $G$  and assign it a color from  $k$  colors  
    end while
```

Spilling: Choose some node in the graph and decide to represent it in memory, not registers!

Example: Need for Spilling (K = 2)

color	register
	eax
	ebx

- What if we get to a state where **all nodes have k or more neighbors** ?

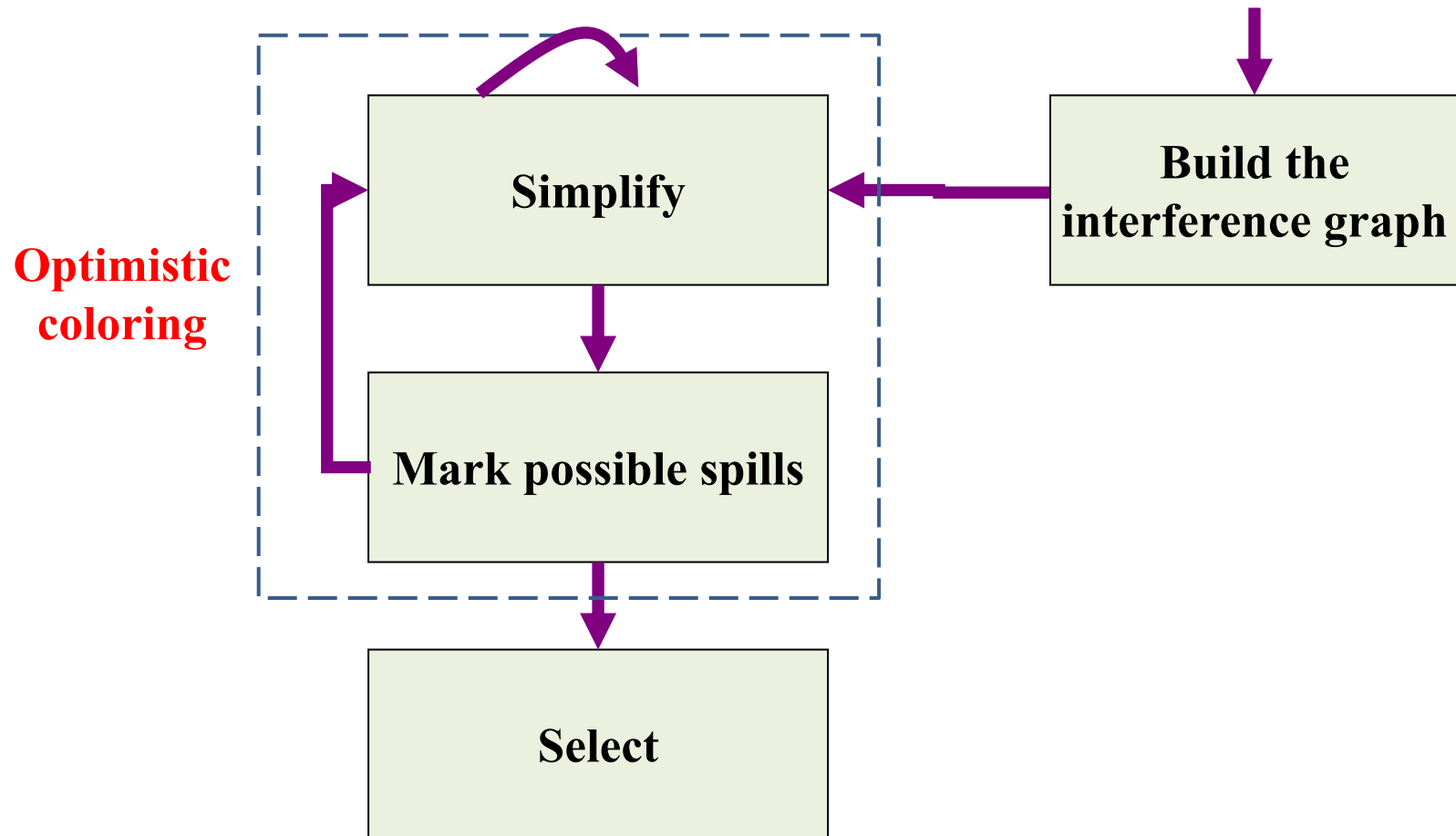


all nodes (a, b, c, e) have 2 neighbours!

Do We Always Need the Actual Spilling?

- At some point, the graph G has nodes only of **significant degree** (that is, nodes of degree $\geq K$).
- **Spilling**: MAY need to choose some node in the graph and decide to represent it in memory
- **Optimistic Coloring** [Chaitin-Briggs Algorithm]
 - An **optimistic approximation** to the effect of spilling: the spilled node does not interfere with any of the other nodes remaining in the graph
 - It can therefore be removed and pushed on the stack, and **the simplify process continued**

Simplify & Select, with Optimistic Coloring

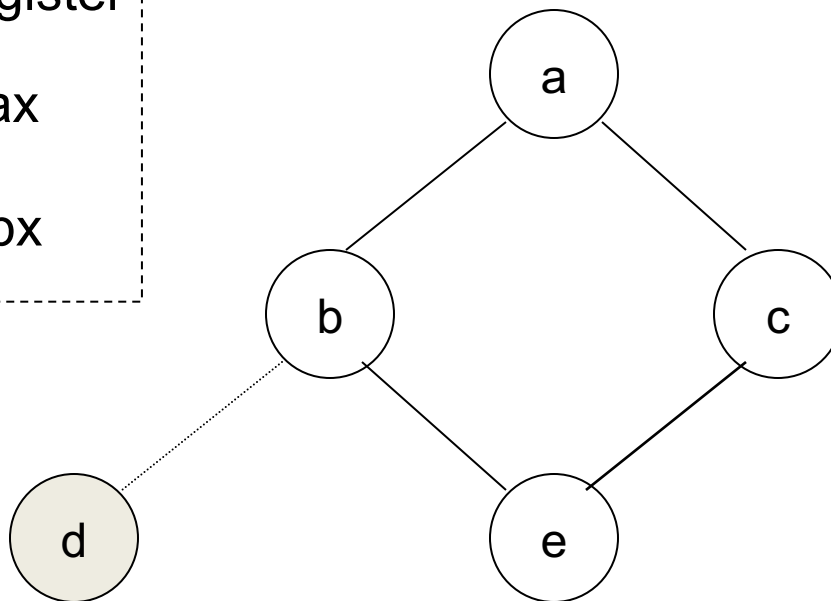


Sometimes, it is unnecessary to do the actual spill! Why?

Example: Optimistic Coloring (K=2)

- Pick a node as a candidate for spilling
 - Remove it from the graph and put it into the stack
 - E.g., we choose **b** and **continue the simplification**

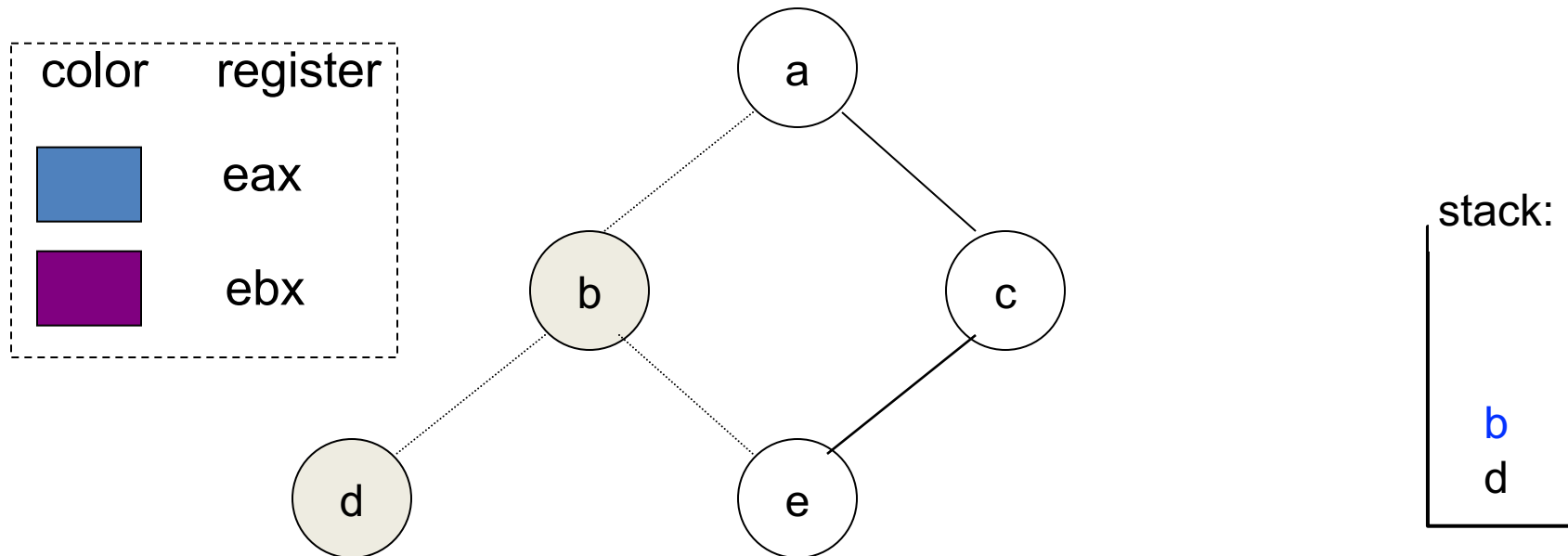
color	register
	eax
	ebx



- 做乐观假设，“照常”把节点删除、放栈上
- 不“打断”simplification策略的正常运行

Example: Optimistic Coloring (K=2)

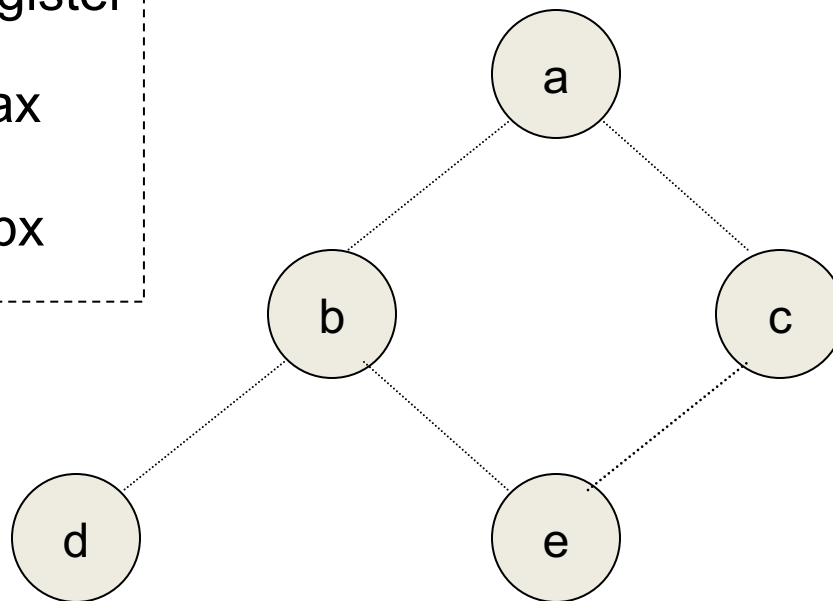
- Pick a node as a candidate for spilling
 - After choosing b, the stack looks as follows



Example: Optimistic Coloring (K=2)

- Continue the simplification process (**a**, **e**, **c**)
- After a few steps, the simplification succeeds (the graph becomes empty)

color	register
	eax
	ebx

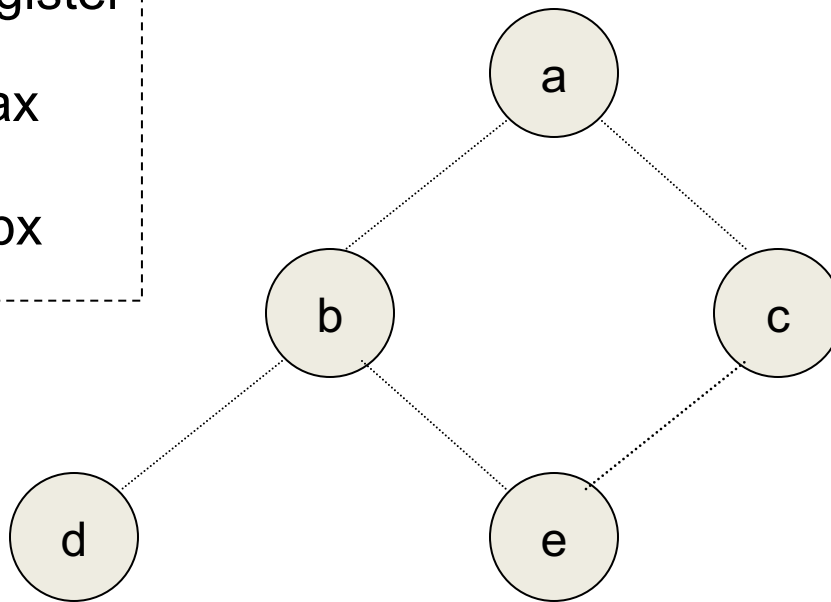


stack:
c
e
a
b
d

Example: Optimistic Coloring (K=2)

- Suppose that the simplification process succeeds (the graph becomes empty)
- We need to start the “select” (coloring) phase

color	register
	eax
	ebx



stack:
c
e
a
b
d

Can we complete the coloring in the select phase as before?

Select (Revised)

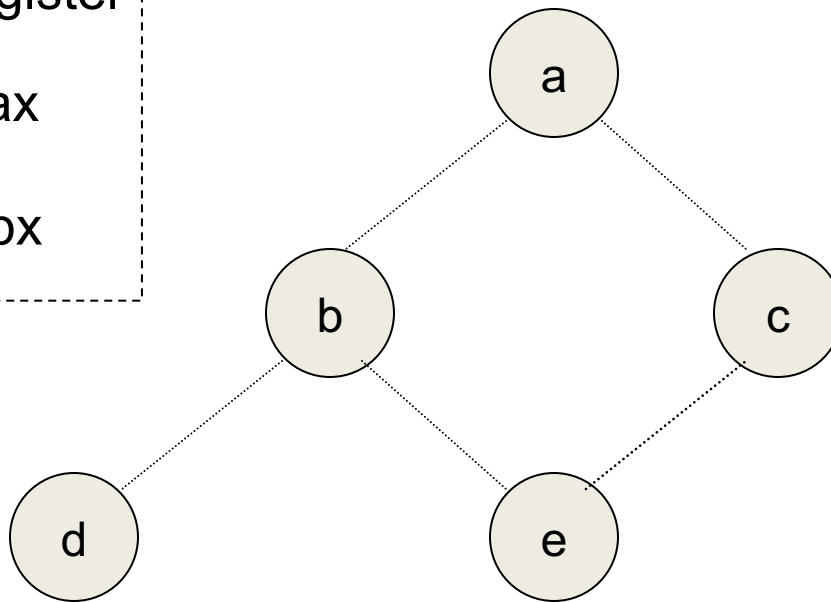
- **Problem:** When *potential spill node* n that was pushed using the **Spill heuristic** is popped, there is no guarantee that it will be colorable.
- **If n 's neighbors are colored with fewer than K colors,** we can color n and n does not become an actual spill.
(The *optimistic coloring works!*)

It is possible that the graph is still K -colorable!

Example: Select Revisited

- Suppose that the simplification process succeeds (the graph becomes empty)
- We need to start the “select” (coloring) phase

color	register
	eax
	ebx



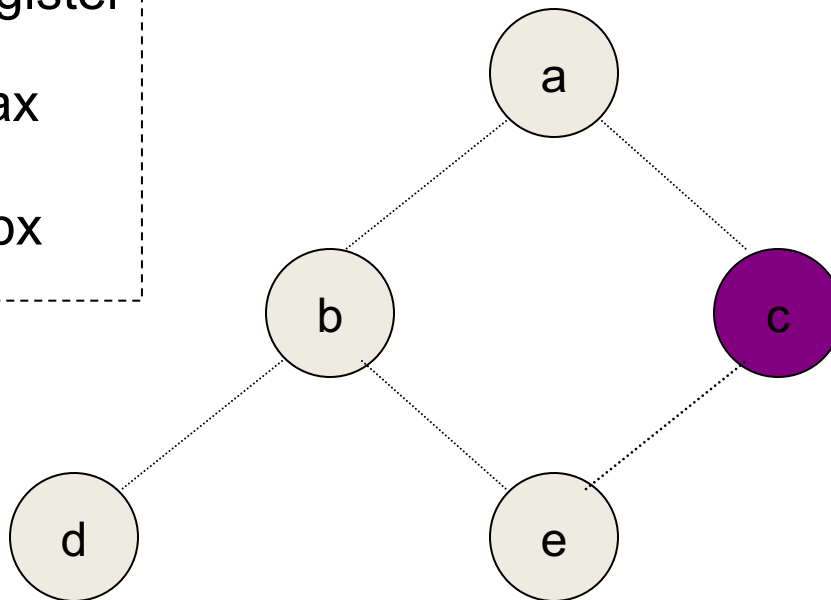
stack:
c
e
a
b
d

Example: Select Revisited

Rebuild and color the graph!

- **Select node c**

color	register
	eax
	ebx



stack:

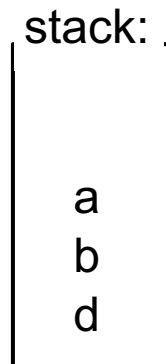
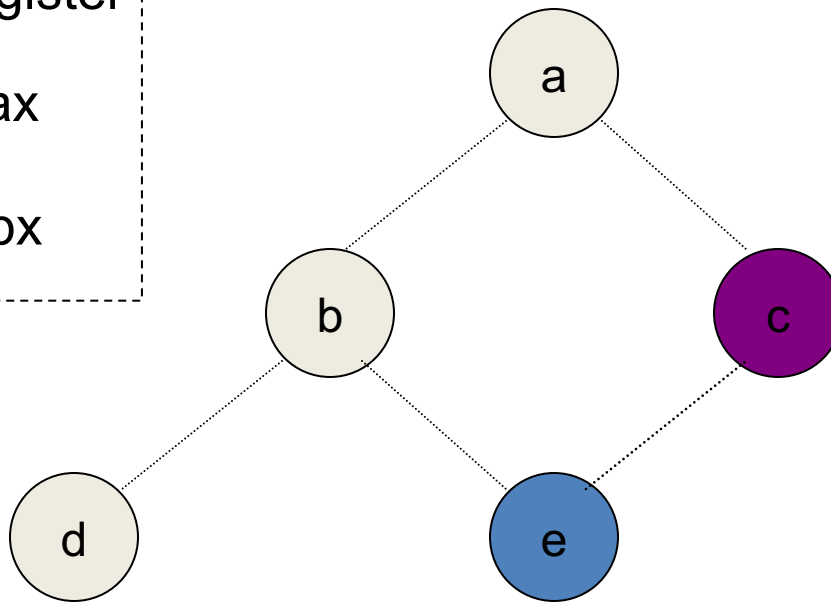
e
a
b
d

Example: Select Revisited

Rebuild and color the graph!

- **Select node e**

color	register
	eax
	ebx

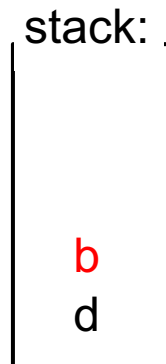
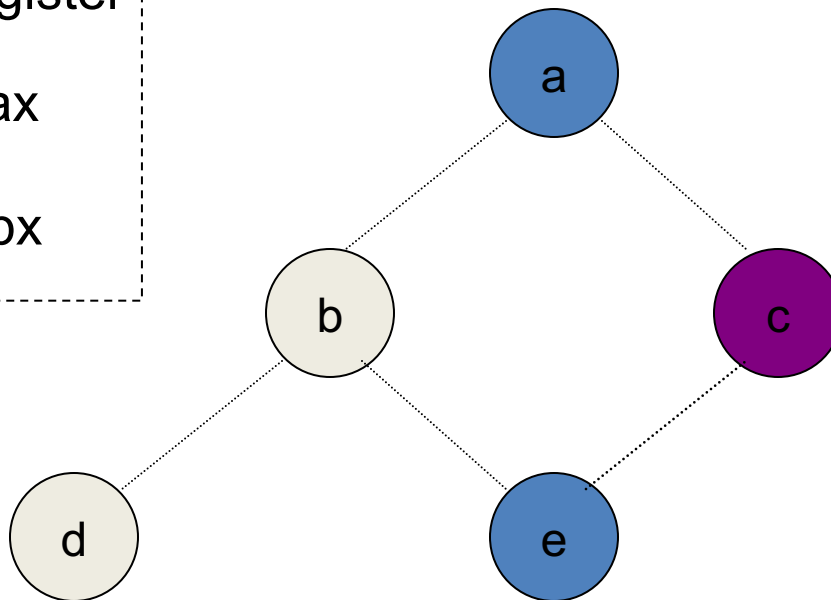


Example: Select Revisited

Rebuild and color the graph!

- **Select node a**

color	register
	eax
	ebx

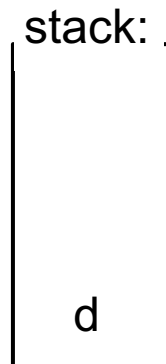
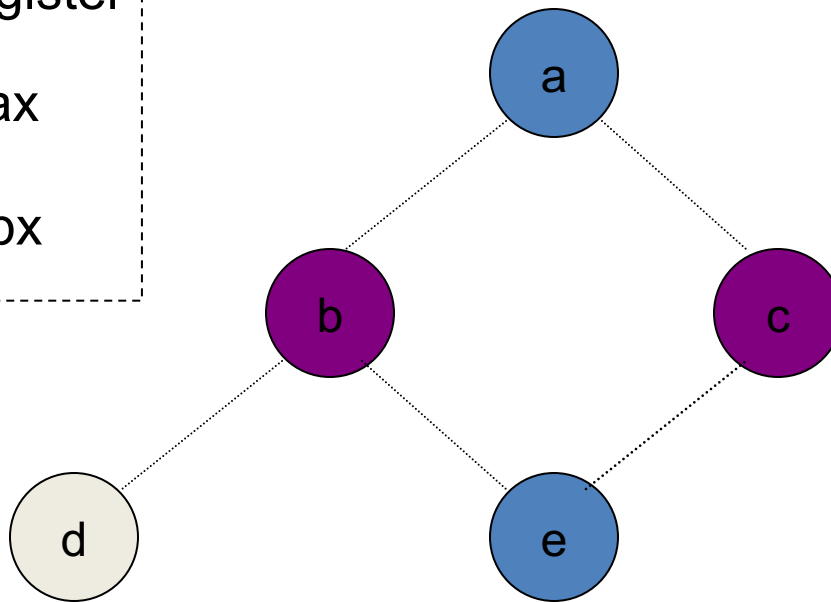


Example: Select Revisited

Rebuild and color the graph!

- **Select node b**

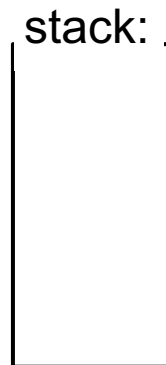
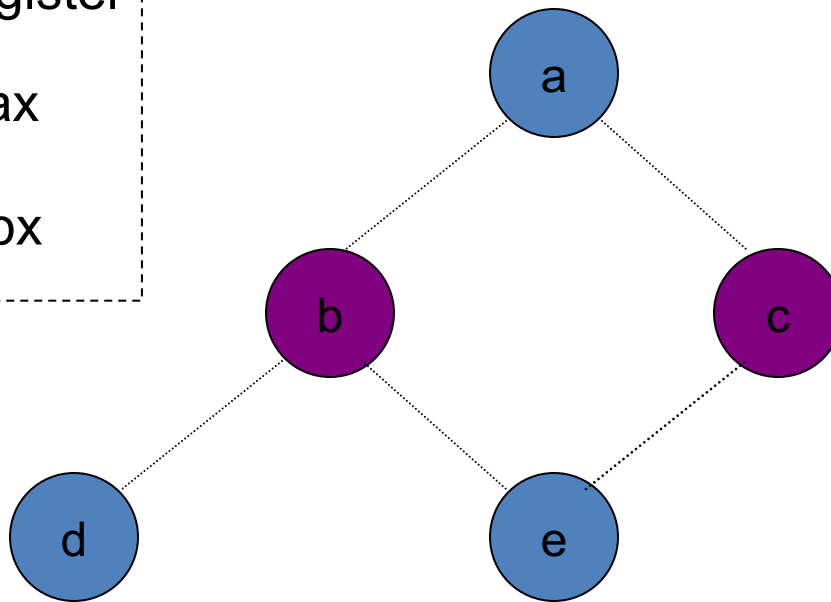
color	register
	eax
	ebx



Example: Select Revisited

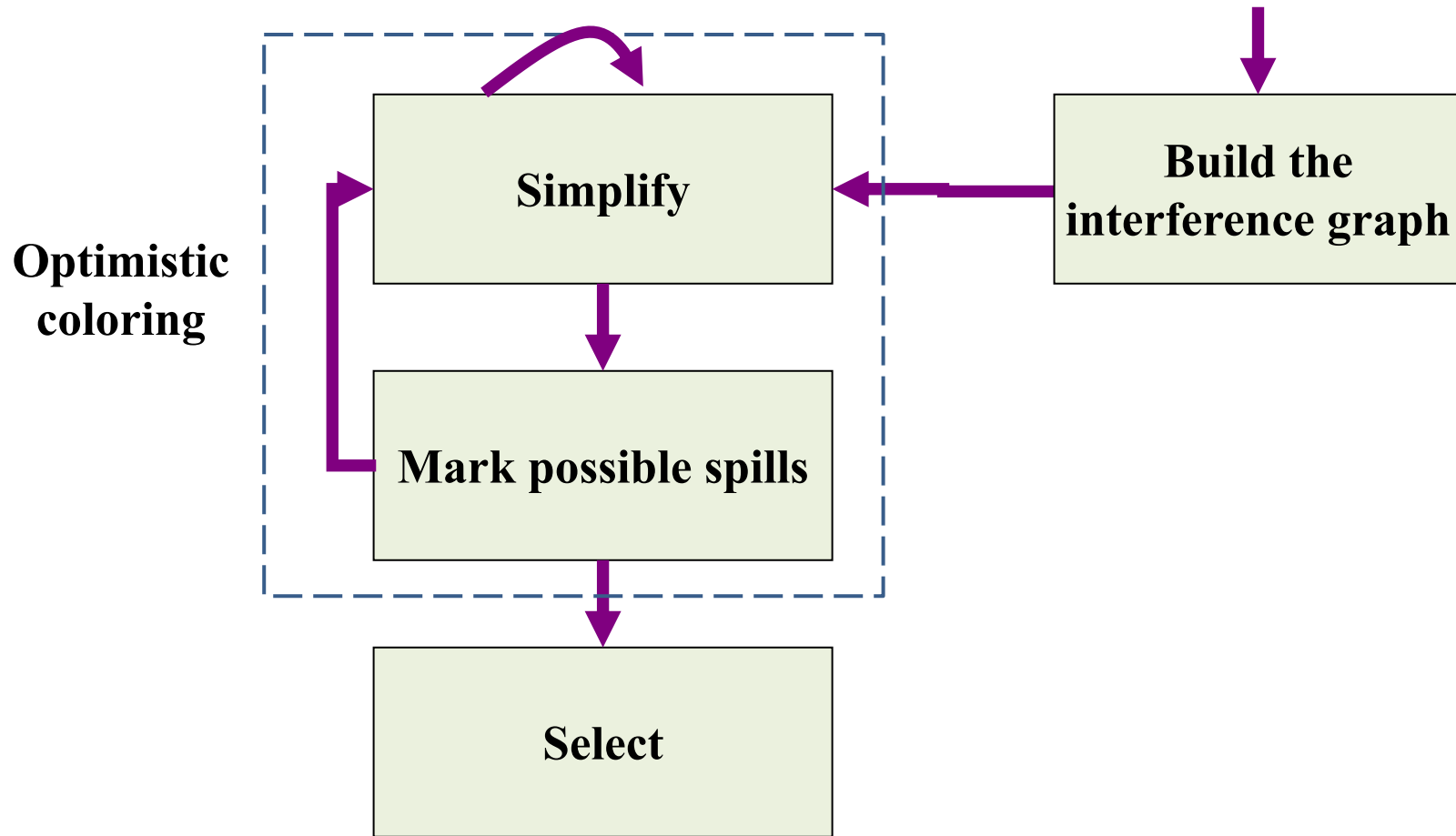
- Rebuild and color the graph!
- **We got lucky**: the “simplify & select” still works!

color	register
	eax
	ebx



Sometimes, it is unnecessary to do the actual spill!

Summary: Simplify & Select, with Optimistic Coloring



Sometimes, it is unnecessary to do the actual spill!

3. Coloring By Simplifications

- **Simplification & Select**
- **Simplification, Select, Spilling**
 - **Optimistic Coloring**
 - **Do the Actual Spill**

Optimistic Heuristic Can Fail!

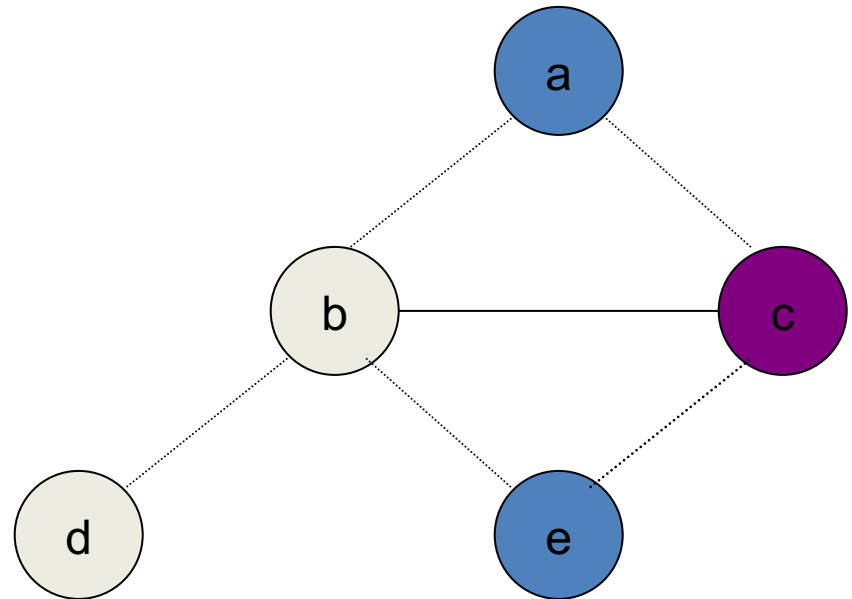
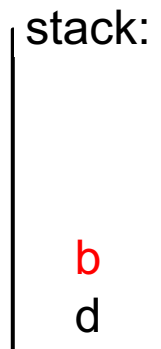
- **Optimistic Coloring** [Chaitin-Briggs Algorithm]
 - An **optimistic approximation** to the effect of spilling: the spilled node does not interfere with any of the other nodes remaining in the graph
 - It can therefore be removed and pushed on the stack, and **the simplify process continued**

However, the optimistic coloring heuristic can fail!

Example: Optimistic Heuristic Fails

- What happens if **no color can be assigned to a marked spilled node** ?
 - How to assign a color to **b** whose neighbors **have 2 different colors already**?! 😞

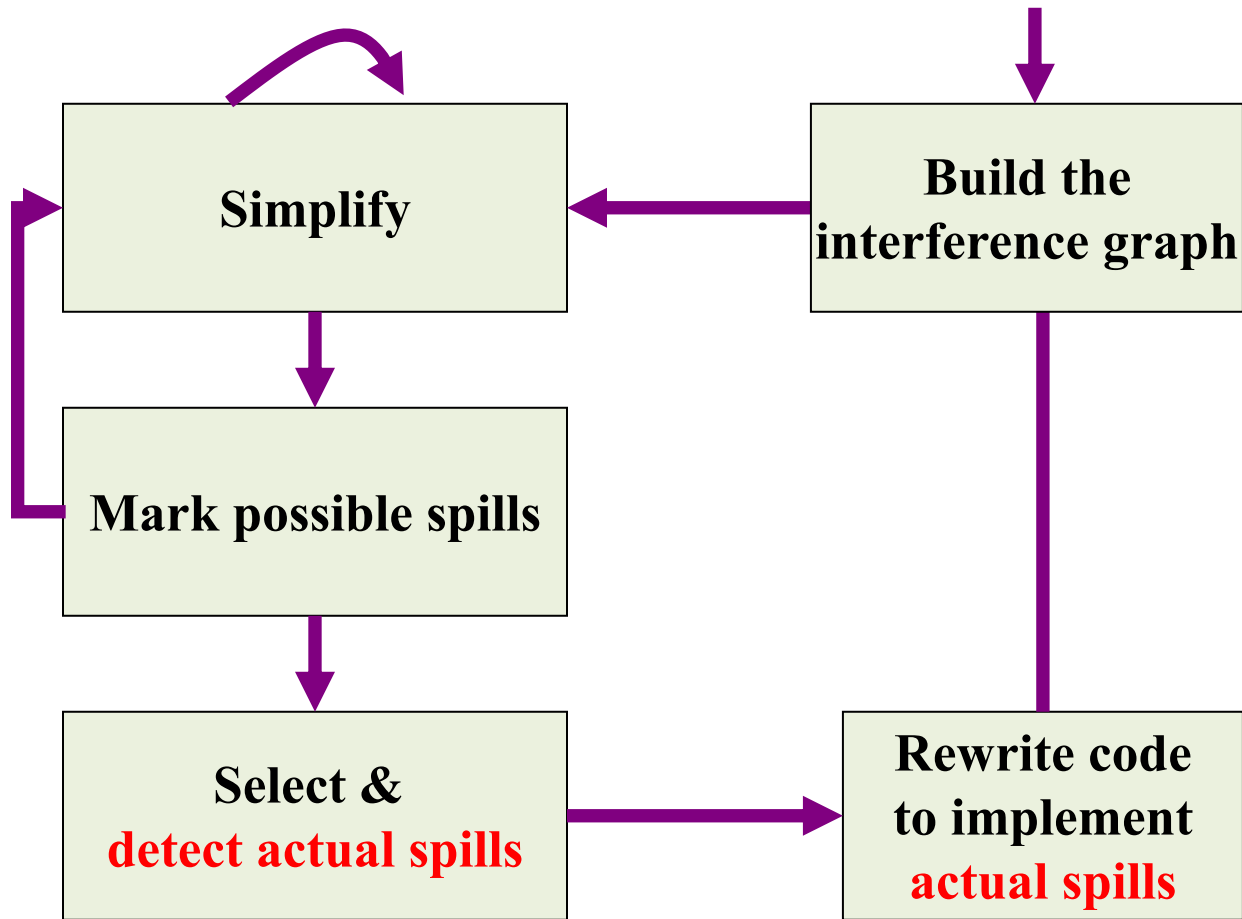
color	register
	eax
	ebx



Select (Revisited)

- **Problem:** When potential spill node **n** that was pushed using the **Spill heuristic** is popped, there is no guarantee that it will be colorable.
 1. If **n**'s neighbors are colored with **fewer than K** colors
 - The *optimistic coloring works*
 2. If **n**'s neighbors have been colored with **K** different colors
 - We have to perform an **actual spill!**
 - We do not assign any color, but continue the **Select** Phase to identify other actual spill

Chaitin's Algorithm



How to Perform the Actual Spill

- 假设变量 c 被选为实际溢出对象：
 - 在栈帧中分配一个内存位置 c_loc 作为 c 的"家"
 - 在每个**使用** c 的指令前插入： $c_i := \text{load } c_loc$
 - 在每个**定义** c 的指令后插入： $\text{store } c_i, c_loc$

Before (原始代码)

```
 $c \leftarrow r3$   
 $a \leftarrow r1$   
...  
 $r3 \leftarrow c$   
return
```

After (溢出后)

```
 $c1 \leftarrow r3$   
 $M[c\_loc] \leftarrow c1$  // store  
 $a \leftarrow r1$   
...  
 $c2 \leftarrow M[c\_loc]$  // load  
 $r3 \leftarrow c2$   
return
```

溢出后原变量 c 被拆成多个**极短生命期**的小变量 ($c1, c2, \dots$)。下一轮着色时它们几乎不与其他变量干涉，通常能顺利分配寄存器。

Example: Actual Spill

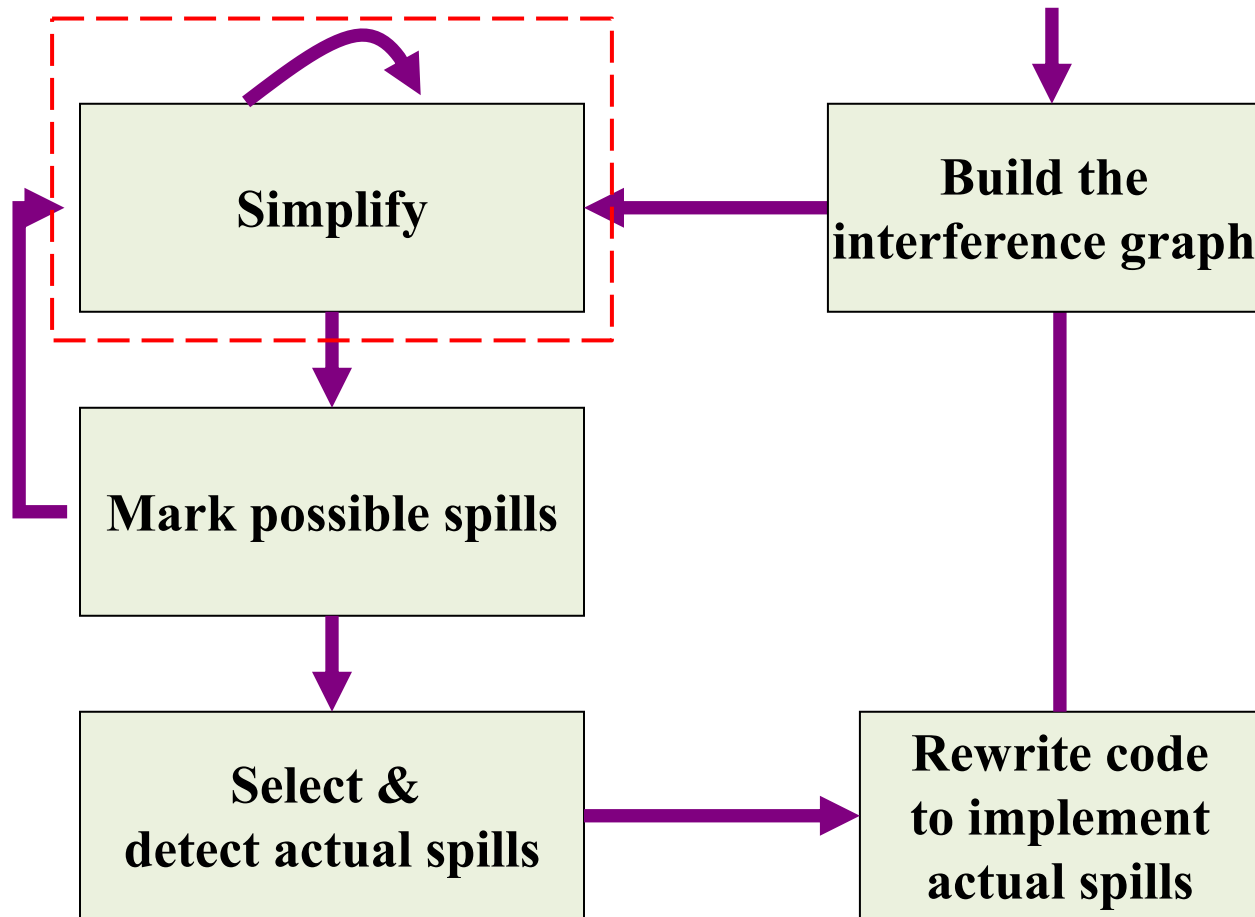
- Suppose **c** was spilled because it had the lowest *spill priority* (to be discussed later.)

```
c ← r3
a ← r1
b ← r2
d ← 0
e ← a
loop: d ← d + b
e ← e - 1
if e > 0 goto loop
r1 ← d
r3 ← c
return
```

```
c1 ← r3
M[cloc] ← c1 // Store c to memory
a ← r1
b ← r2
d ← 0
e ← a
loop: d ← d + b
e ← e - 1
if e > 0 goto loop
r1 ← d
c2 ← M[cloc] // Fetch c from memory
r3 ← c2
return
```

- The spilled variable **c** is turned into multiple temporaries (**c1**, **c2**) with very small live ranges!
- ➔ Creates more opportunities for register allocation in the next round.

Summary: “Simplification → Select” Revised



Example (K = 4)

- Can start with the nodes **g**, **h**, **c**, and **f** in its working set (they have less than four neighbors each)

Live in: k j

g := mem[j+12]

h := k-1

f := g*h

e := mem[j+8]

m := mem[j+16]

b := mem[f]

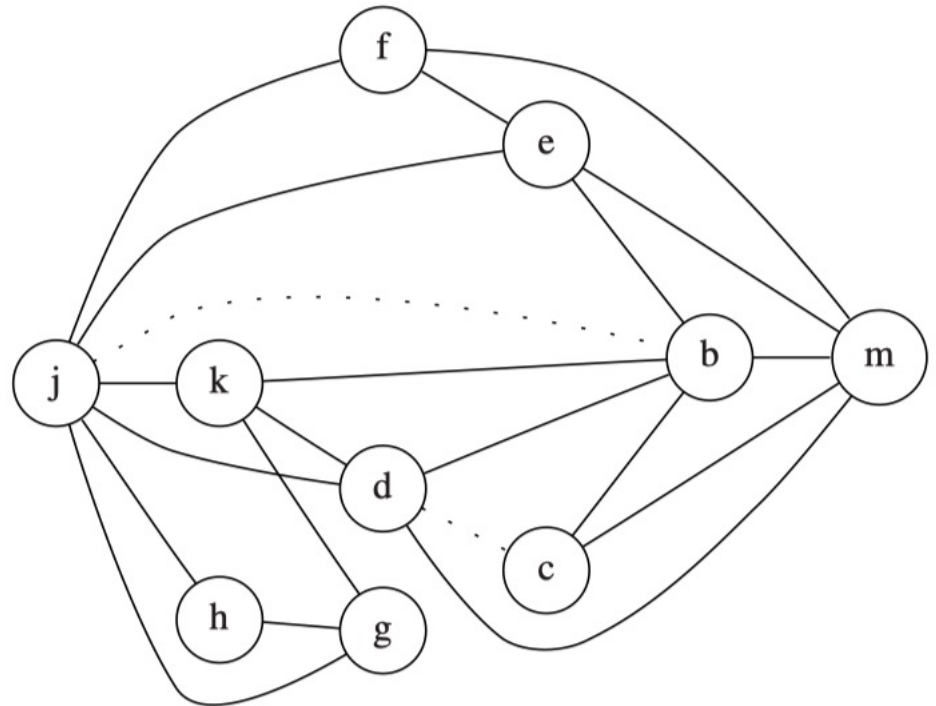
c := e+8

d := c

k := m+4

j := b

Live out: d k j

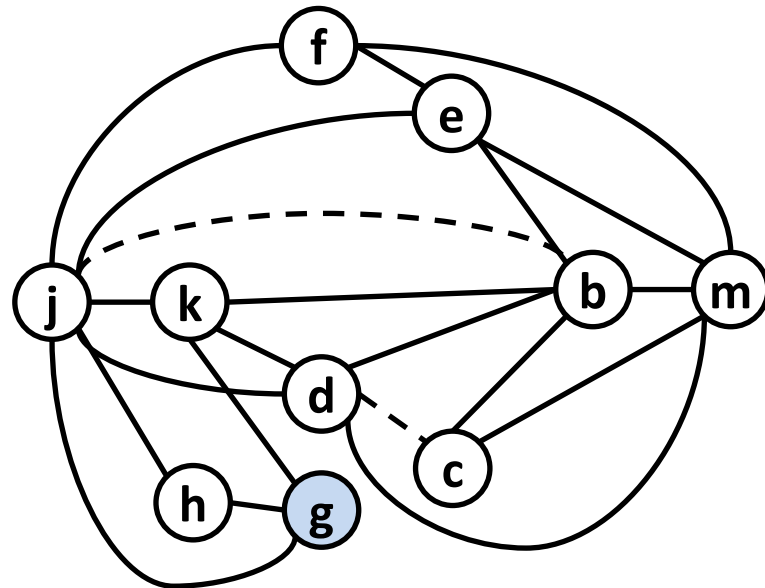


Dotted lines indicate move instructions

Example (K=4): Simplify Phase

- Remove **g** ($\deg(g)=2 < 4$)

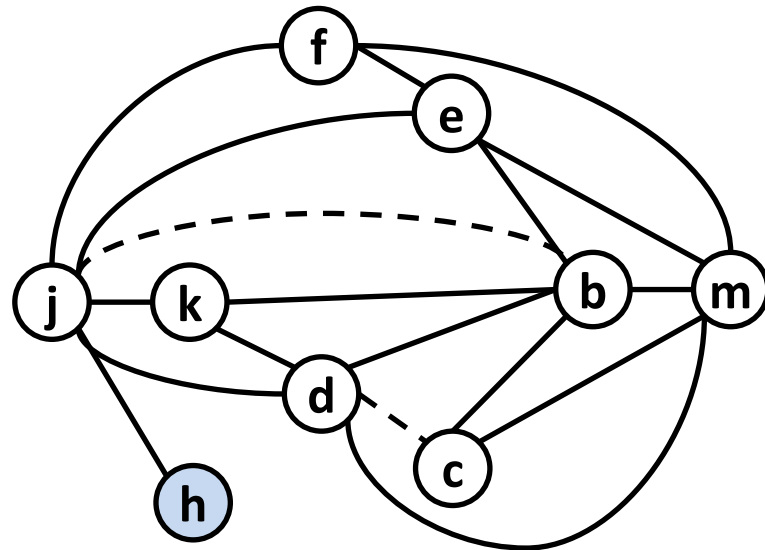
Stack	Assignment



Example (K=4): Simplify Phase

- Remove **h** ($\deg(h)=1 < 4$)

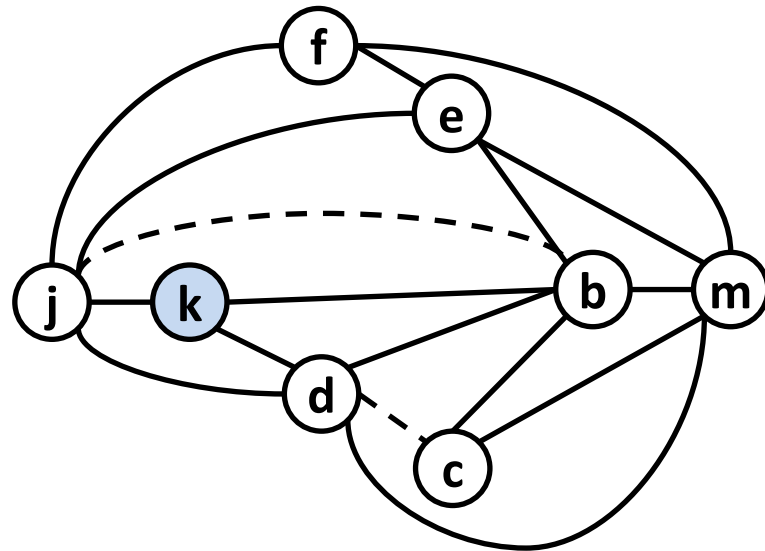
g	
Stack	Assignment



Example (K=4): Simplify Phase

- Remove **k**

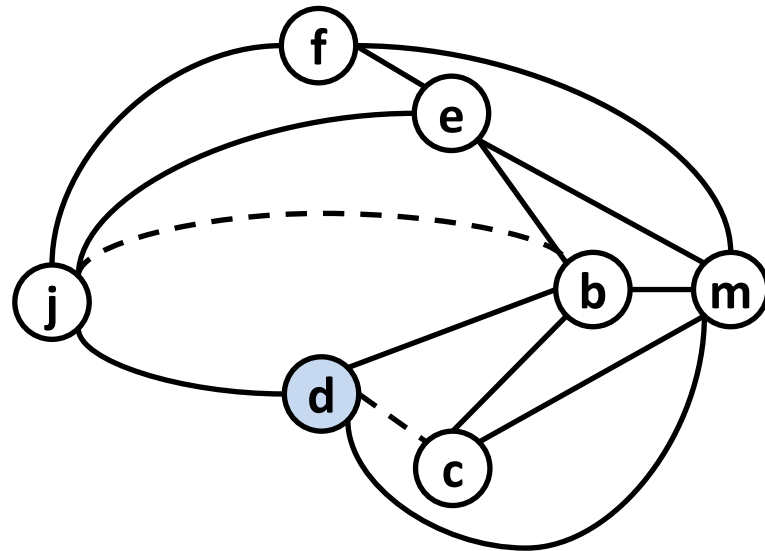
h	
g	
Stack	Assignment



Example (K=4): Simplify Phase

- Remove **d**

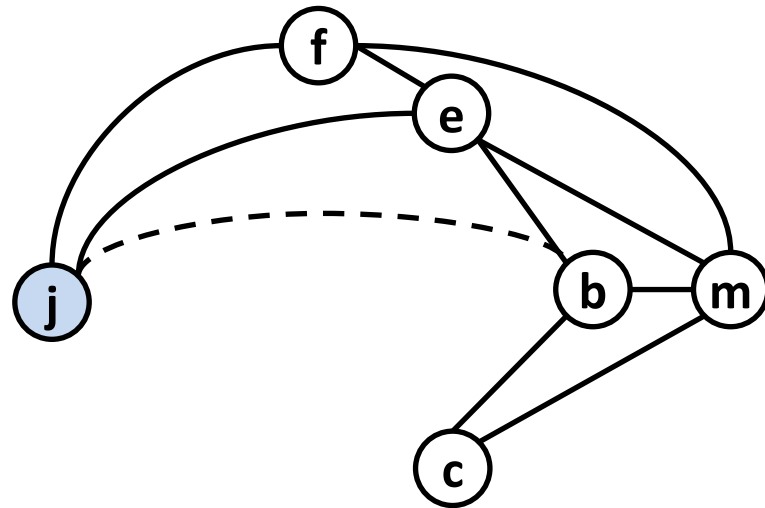
k	
h	
g	
Stack	Assignment



Example (K=4): Simplify Phase

- Remove **j**

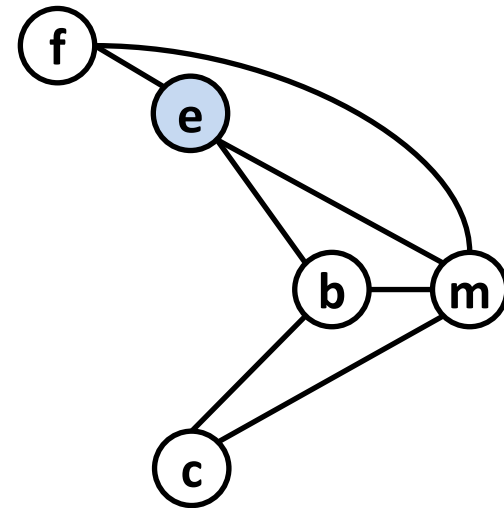
d	
k	
h	
g	
Stack	Assignment



Example (K=4): Simplify Phase

- Remove **e**

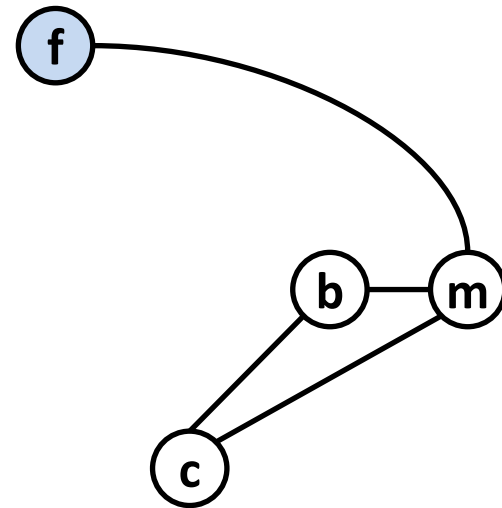
j	
d	
k	
h	
g	
Stack	Assignment



Example (K=4): Simplify Phase

- Remove **f**

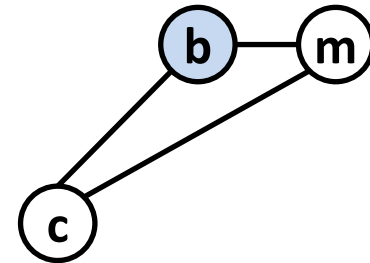
e	
j	
d	
k	
h	
g	
Stack	Assignment



Example (K=4): Simplify Phase

- Remove **b**

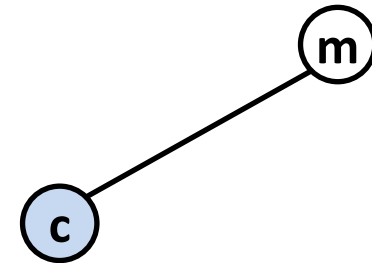
f	
e	
j	
d	
k	
h	
g	
Stack	Assignment



Example (K=4): Simplify Phase

- Remove **c**

b	
f	
e	
j	
d	
k	
h	
g	
Stack	Assignment



Example (K=4): Simplify Phase

- Remove **m**

c	
b	
f	
e	
j	
d	
k	
h	
g	
Stack	Assignment

m

Example (K=4): Simplify Phase

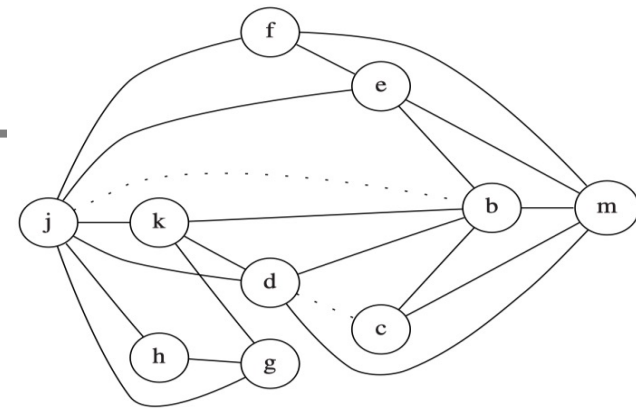
- The simplification succeeds!

m	
c	
b	
f	
e	
j	
d	
k	
h	
g	
Stack	Assignment

步骤	操作	理由	栈 (顶→底)
1	删除 g	$\deg(g)=2 < 4$	g
2	删除 h	$\deg(h)=1 < 4$	h, g
3	删除 k	$\deg(k)$ 降至 $3 < 4$	k, h, g
4	删除 d	$\deg(d)$ 降至 $2 < 4$	d, k, h, g
5	删除 j	$\deg(j)$ 降至 $2 < 4$	j, d, k, h, g
6	删除 e	$\deg(e)$ 降至 $3 < 4$	e, j, d, k, h, g
7	删除 f	$\deg(f)$ 降至 $2 < 4$	f, e, j, d, k, h, g
8	删除 b	$\deg(b)=2 < 4$	b, f, e, j, d, k, h, g
9	删除 c	$\deg(c)=1 < 4$	c, b, f, e, j, d, k, h, g
10	删除 m	$\deg(m)=0$	m, c, b, f, e, j, d, k, h, g

Example (K = 4): Select

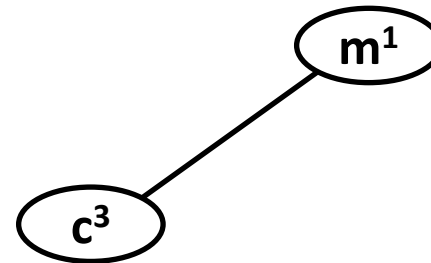
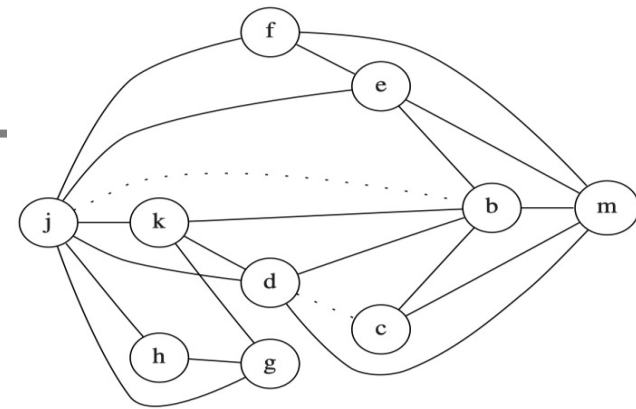
m	1
c	
b	
f	
e	
j	
d	
k	
h	
g	
Stack	Assignment



m^1

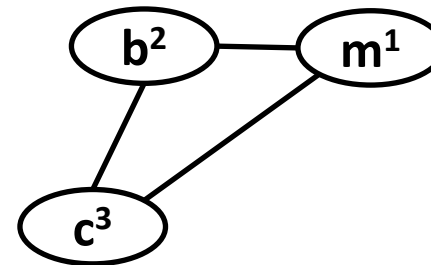
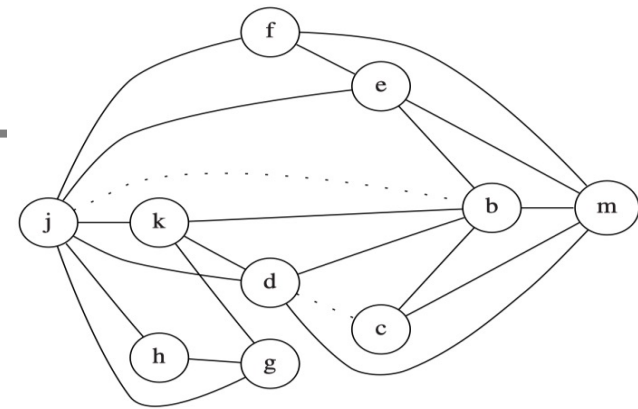
Example (K = 4): Select

m	1
c	3
b	
f	
e	
j	
d	
k	
h	
g	
Stack	Assignment



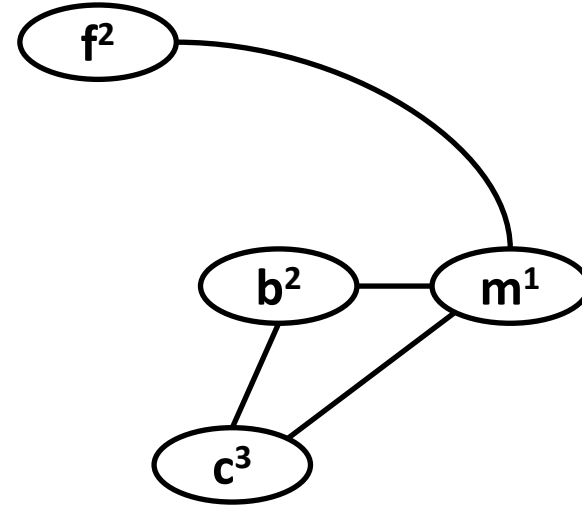
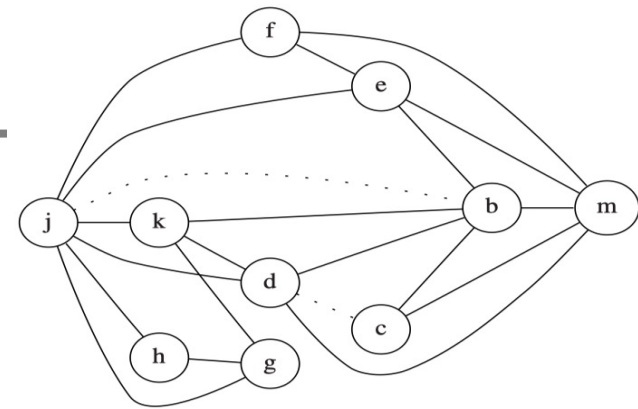
Example (K = 4): Select

m	1
c	3
b	2
f	
e	
j	
d	
k	
h	
g	
Stack	Assignment



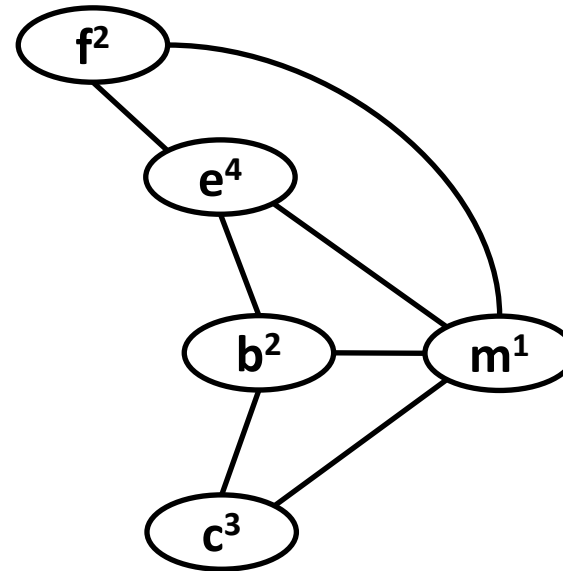
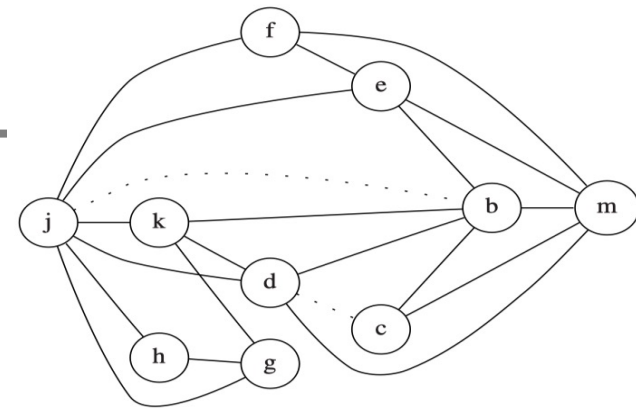
Example (K = 4): Select

m	1
c	3
b	2
f	2
e	
j	
d	
k	
h	
g	
Stack	Assignment



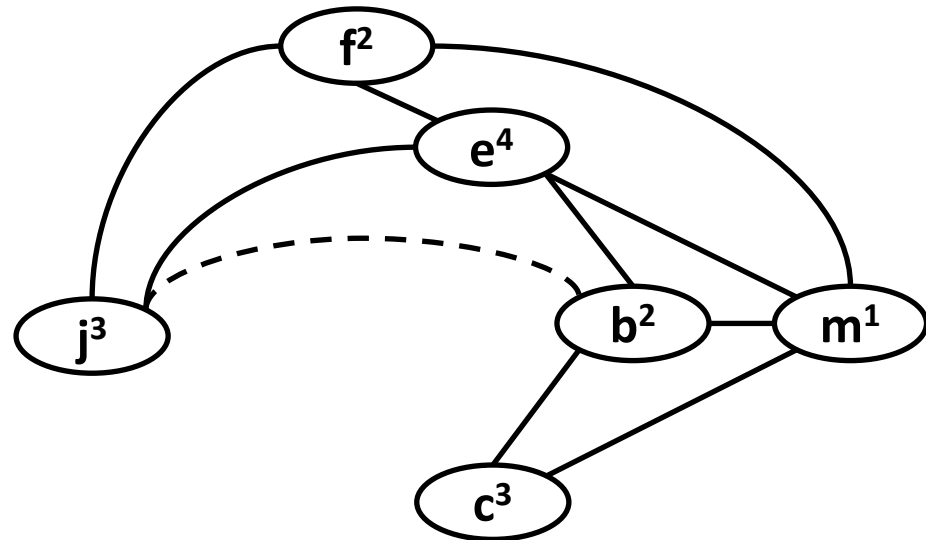
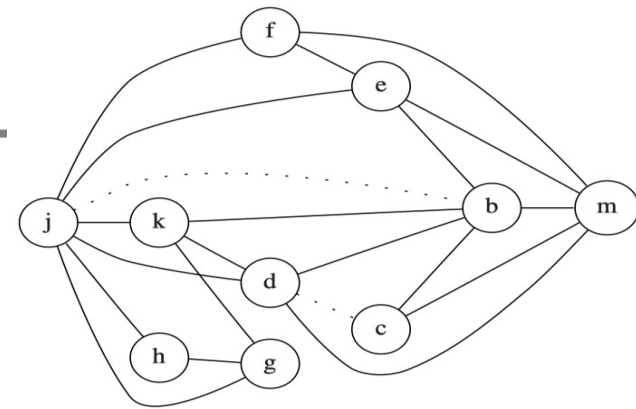
Example (K = 4): Select

m	1
c	3
b	2
f	2
e	4
j	
d	
k	
h	
g	
Stack	Assignment



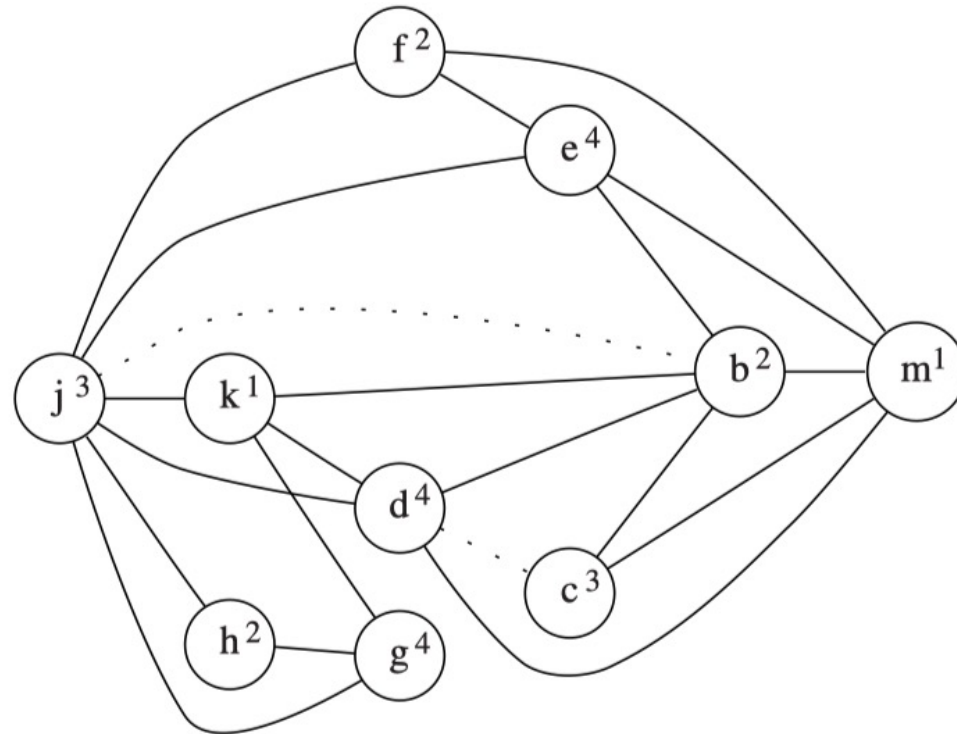
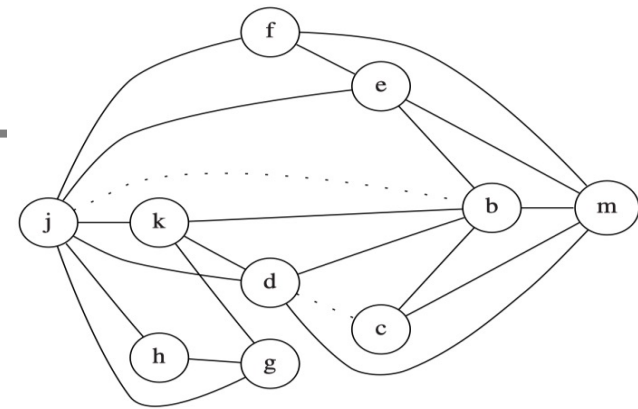
Example (K = 4): Select

m	1
c	3
b	2
f	2
e	4
j	3
d	
k	
h	
g	
Stack	Assignment



Example (K = 4): Select

m	1
c	3
b	2
f	2
e	4
j	3
d	4
k	1
h	2
g	4
Stack	Assignment



Example (K = 4)

Live in: k j

g := mem[j+12]

h := k-1

f := g*h

e := mem[j+8]

m := mem[j+16]

b := mem[f]

c := e+8

d := c

k := m+4

j := b

Live out: d k j



Live in: k j

r4 := mem[r3+12]

r2 := r1-1

r2 := r4*r2

r4 := mem[r3+8]

r1 := mem[r3+16]

r2 := mem[r2]

r3 := r4+8

r4 := r3

r1 := r1+4

r3 := r2

Live out d k j

红色 move 指令未能消除——d≠c
且 j≠b 的颜色不同。
后续 coalescing 优化可尝试消除。

Summary

概念	要点
寄存器分配	∞ 虚拟寄存器 $\rightarrow$ K 个物理寄存器的映射
干涉图	节点=temp, 边=生命期重叠的变量对
图着色	K-着色  合法的 K 寄存器分配
构建规则	Non-move: 加边到所有 live-out; Move: 跳过 source
Move Coalescing	合并 source & dest $\rightarrow$ 消除 copy 指令
Kempe 定理	度 $< K$ 的节点总可着色 $\rightarrow$ 基于栈的算法
Simplify & Select	删除低度节点入栈 $\rightarrow$ 逆序着色
Optimistic Coloring	溢出候选先乐观入栈, 着色时再决定
Actual Spill	无法着色 $\rightarrow$ 插入 load/store $\rightarrow$ 重新着色

4. Coloring By Simplifications, To Be Continued...

Coming Next ...

- Coalescing heuristics (George's criterion, Briggs' criterion)
- Freeze phase
- Precolored nodes
- Iterated Register Coalescing (IRC) — 完整算法
 - Spill priority / cost metrics



Thank you all for your attention